

ĐẠI HỌC QUỐC GIA TP HCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN
23TNT1

Đồ án 2

Đề tài: Khai thác tập phổ biến tối đại bằng FP-Max

Môn học: Khai thác dữ liệu và ứng dụng

Sinh viên thực hiện:

22120457 - Khưu Hải Châu

23122006 - Lưu Thượng Hồng

23122020 - Nguyễn Thiên Ân

23122034 - Lê Nguyên Khang

23122036 - Nguyễn Ngọc Khoa

Giáo viên hướng dẫn:

GS.TS. Lê Hoài Bắc

ThS. Lê Nhật Nam

14 tháng 6, 2026



Mục lục

1	Giới thiệu	1
2	Phân công công việc	2
3	Khái niệm và ký hiệu	2
3.1	Bảng ký hiệu	2
3.2	Các khái niệm chính	3
4	Nền tảng lý thuyết	4
4.1	Bài toán Frequent Itemset Mining	4
4.2	Bộ máy nền: FP-Growth và FP-tree	5
4.3	Ý tưởng và thuật toán FP-Max	6
4.4	Phân tích độ phức tạp	7
4.5	Vị trí lịch sử	9
5	Ví dụ minh họa	10
5.1	Ví dụ 1: CSDL cơ sở	10
5.2	Ví dụ 2: Tỉa theo siêu tập	11
6	Cài đặt	13
6.1	Môi trường và tổ chức mã nguồn	13
6.2	Cài đặt FP-Growth cơ bản	14
6.3	Cài đặt FP-Growth tối ưu	15
6.4	Tránh performance anti-patterns	16
6.5	Cài đặt FP-Max và baseline naive-maximal	17
6.6	Kiểm thử tự động	18
6.7	Xử lý đầu vào và đầu ra	18
7	Thực nghiệm và đánh giá	19
7.1	Thiết lập thực nghiệm	19
7.2	Tập dữ liệu benchmark	19
7.3	Tính đúng đắn	20

7.4	Thời gian chạy theo minsup	20
7.5	Số lượng tập tối đại theo minsup	25
7.6	Bộ nhớ	27
7.7	Khả năng mở rộng	28
7.8	Ảnh hưởng của độ dài giao dịch	29
7.9	Nhận xét và hướng tối ưu tiếp theo	30
8	Ứng dụng	31
8.1	Bài toán phân tích giỏ hàng	31
8.2	Dữ liệu và thiết lập	31
8.3	Top-10 luật theo lift	32
8.4	Diễn giải kinh doanh	32
9	Kết luận	33
	Tài liệu	35
A	Phụ lục: Hướng dẫn tái lập	36
A.1	Cấu trúc thư mục	36
A.2	Môi trường và cài đặt	36
A.3	Chạy chương trình	37
A.4	Kiểm thử	37
A.5	Tái lập thực nghiệm Chương 4	38
A.6	Tái lập ứng dụng Chương 5	38
A.7	Tính tái lập	39

Danh sách bảng

1	Bảng phân công công việc của nhóm	2
2	Bảng ký hiệu dùng xuyên suốt báo cáo.	3
3	Độ phức tạp thời gian của FP-Max theo từng trường hợp.	8
4	Độ phức tạp không gian của FP-Max theo từng trường hợp.	9
5	CSDL toy dùng để minh hoạ FP-Max.	10
6	Các giao dịch sau bước tỉa và sắp thứ tự để dựng FP-tree.	10
7	Vòng đệ quy FP-Max trên ví dụ cơ sở. Mỗi candidate được hợp nhất vào danh sách MFI theo quy tắc loại tập con nghiêm ngặt.	11
8	CSDL thiết kế để kích hoạt phép tỉa theo siêu tập trong FP-Max.	12
9	Vòng đệ quy FP-Max trên CSDL Bảng 8. Hai item cuối bị tỉa nhờ kiểm tra siêu tập.	12
10	Bộ nhớ cấp phát của baseline naive-maximal và FP-Max trên Chess, đo bằng <code>alloc_bytes</code>	18
11	Các tập dữ liệu benchmark dùng trong Mục 7.	20
12	Kết quả correctness đại diện khi so số tập tối đại với SPMF FPMax.	20
13	Peak RSS thật (<code>Sys.maxrss</code>) đo trong tiến trình riêng cho mỗi lần chạy. Cột <i>sản</i> là cấu hình chỉ load dữ liệu, không mining.	28
14	Ảnh hưởng của độ dài giao dịch trung bình lên thời gian chạy FP-Max.	29
15	Top-10 luật kết hợp trên Groceries theo lift, với $\text{minsup} = 0.01$ và $\text{minconf} = 0.2$	32

Danh sách hình vẽ

1	FP-tree minh hoạ cho CSDL toy ở Mục 5 với $\text{minsup} = 0.6$	6
2	Thời gian chạy theo minsup trên Chess.	21
3	Thời gian chạy theo minsup trên Mushrooms.	22
4	Thời gian chạy theo minsup trên Retail.	23
5	Thời gian chạy theo minsup trên T10I4D100K.	24
6	Thời gian chạy theo minsup trên Accidents.	25
7	Số tập tối đại theo minsup trên Chess.	26
8	Số tập tối đại theo minsup trên Retail.	26
9	Bộ nhớ cấp phát của baseline naive-maximal và FP-Max tại minsup trung bình. . .	27
10	Khả năng mở rộng theo số lượng giao dịch.	29
11	Thời gian chạy FP-Max khi tăng độ dài giao dịch trung bình.	30

1 Giới thiệu

Khai thác tập phổ biến (Frequent Itemset Mining, FIM) là một bài toán nền tảng trong khai thác dữ liệu giao dịch. Với một cơ sở dữ liệu gồm nhiều giao dịch, mục tiêu của FIM là tìm các nhóm item xuất hiện cùng nhau đủ thường xuyên theo một ngưỡng hỗ trợ tối thiểu. Kết quả của bài toán này thường được dùng làm đầu vào cho luật kết hợp, phân tích giỏ hàng, phát hiện mẫu trong log hệ thống, hoặc phân tích các tổ hợp đặc trưng trong dữ liệu sinh học.

Trong đồ án này, nhóm lựa chọn thuật toán FP-Max (FPmax) để khai thác tập phổ biến tối đại (maximal frequent itemset) [Grahne and Zhu \(2003\)](#). FP-Max được xây dựng trên cấu trúc FP-tree của FP-Growth: FP-Growth nén cơ sở dữ liệu vào một cây tiền tố và khai thác bằng các cơ sở mẫu điều kiện để tránh bước sinh ứng viên tốn kém của Apriori [Han et al. \(2000\)](#), còn FP-Max dùng lại chính cây này nhưng chỉ giữ lại các tập phổ biến tối đại nhờ một danh sách MFI và phép tỉa theo siêu tập (superset pruning), thay vì liệt kê toàn bộ tập phổ biến. Trên cơ sở đó, nhóm cài đặt ba thành phần: cây FP-tree cùng bộ máy khai thác FP-Growth (bản cơ bản và bản tối ưu hoá bộ nhớ/tốc độ) làm nền, một baseline maximal kiểu *naive* (khai thác toàn bộ tập phổ biến rồi lọc tối đại), và FP-Max khai thác trực tiếp có tỉa. Cài đặt chính được thực hiện bằng Julia, có giao diện dòng lệnh, bộ kiểm thử tự động, và harness thực nghiệm so sánh với cài đặt FPMMax tham chiếu trong SPMF [Fournier-Viger et al. \(2014\)](#).

Báo cáo được tổ chức như sau. Mục [4](#) trình bày các định nghĩa hình thức của FIM, tính chất Apriori, cấu trúc FP-tree, giả mã và phân tích độ phức tạp của FP-Max. Mục [5](#) minh hoạ FP-Max bằng hai ví dụ tay: một ví dụ cơ sở chạy đầy đủ vòng đệ quy và cập nhật danh sách MFI, và một trường hợp đặc biệt cho thấy phép tỉa theo siêu tập bỏ qua nguyên một nhánh. Mục [6](#) mô tả thiết kế cài đặt, các cấu trúc dữ liệu, tối ưu hoá và cách chạy chương trình. Mục [7](#) trình bày thực nghiệm về tính đúng đắn so với SPMF FPMMax, thời gian chạy, số lượng tập tối đại, bộ nhớ, khả năng mở rộng và ảnh hưởng của độ dài giao dịch. Mục [8](#) áp dụng thuật toán vào phân tích giỏ hàng trên tập Groceries và sinh luật kết hợp từ các tập phổ biến tìm được.

2 Phân công công việc

Bảng 1: Bảng phân công công việc của nhóm

MSSV	Họ và tên	Công việc
22120457	Khưu Hải Châu	Nghiên cứu FP-Max, FP-tree prerequisite và viết Chương 1
23122006	Lưu Thượng Hồng	Cài đặt FP-tree/FP-Growth nền, parser, I/O và CLI Julia
23122020	Nguyễn Thiên Ân	Cài đặt FP-Max, tối ưu FP-Growth engine, unit test và đối chiếu SPMF
23122034	Lê Nguyên Khang	Thiết kế thực nghiệm, đo thời gian/bộ nhớ và sinh biểu đồ
23122036	Nguyễn Ngọc Khoa	Ứng dụng Groceries, tổng hợp báo cáo và kiểm tra PDF

3 Khái niệm và ký hiệu

Phần này tổng hợp các khái niệm và ký hiệu được dùng nhất quán xuyên suốt báo cáo, nhằm tránh lặp lại định nghĩa ở từng phần. Các định nghĩa hình thức đầy đủ được trình bày ở Mục 4; ở đây chỉ liệt kê để tiện tra cứu. Trừ khi nói rõ là tương đối, mọi giá trị support trong báo cáo đều là support tuyệt đối (đếm số giao dịch).

3.1 Bảng ký hiệu

Bảng 2 liệt kê các ký hiệu toán học chính và ý nghĩa của chúng.

Bảng 2: Bảng ký hiệu dùng xuyên suốt báo cáo.

Ký hiệu	Ý nghĩa
$I = \{i_1, \dots, i_m\}$	Tập tất cả item; m là số item phân biệt
$D = \{T_1, \dots, T_n\}$	Cơ sở dữ liệu giao dịch; $n = D $ là số giao dịch
T_j	Giao dịch thứ j , với $T_j \subseteq I$
X, Y	Itemset, tức tập con của I
$X \subseteq T_j$	Giao dịch T_j chứa itemset X
$\text{sup}(X)$	Support tuyệt đối: số giao dịch trong D chứa X
$\text{sup}_{rel}(X)$	Support tương đối: $\text{sup}(X)/ D $
θ	Ngưỡng support tối thiểu tương đối (minsup)
$\lceil \theta n \rceil$	Ngưỡng support tối thiểu tuyệt đối tương ứng
$X \Rightarrow Y$	Luật kết hợp, với $X \cap Y = \emptyset$
$\text{conf}(X \Rightarrow Y)$	Confidence: $\text{sup}(X \cup Y) / \text{sup}(X)$
$\text{lift}(X \Rightarrow Y)$	Lift: $\text{conf}(X \Rightarrow Y) / \text{sup}_{rel}(Y)$
minconf	Ngưỡng confidence tối thiểu của luật
$\bar{e}$	Độ dài giao dịch trung bình sau bước tủa item không phổ biến
L	Tổng số item còn lại sau tủa trên toàn bộ giao dịch

3.2 Các khái niệm chính

Các thuật ngữ sau xuất hiện lặp lại trong các chương sau và được hiểu thống nhất như dưới đây:

- **Tập phổ biến (frequent itemset):** itemset X với $\text{sup}_{rel}(X) \geq \theta$, tương đương $\text{sup}(X) \geq \lceil \theta n \rceil$ Agrawal et al. (1993).
- **Tập đóng (closed itemset):** tập phổ biến X không có siêu tập phổ biến nào cùng support.
- **Tập tối đại (maximal itemset):** tập phổ biến X không có siêu tập phổ biến nào; mọi tập tối đại đều là tập đóng.
- **Luật kết hợp (association rule):** suy diễn $X \Rightarrow Y$ được đánh giá qua support, confidence và lift Agrawal et al. (1993).
- **FP-tree:** cây nén tiền tố biểu diễn cô đọng cơ sở dữ liệu giao dịch; mỗi node lưu item, biến đếm *count*, con trỏ cha và tập con Han et al. (2000).
- **Header table:** bảng ánh xạ mỗi item tới danh sách các node mang item đó trong FP-tree.

- **Conditional pattern base:** tập các đường tiền tố kết thúc tại một item, dùng để dựng cây điều kiện.
- **Conditional FP-tree:** FP-tree xây từ conditional pattern base của một hậu tố, dùng trong khai thác đệ quy.
- **Single-path:** trường hợp FP-tree (hoặc cây điều kiện) chỉ có một đường đi, cho phép sinh trực tiếp mọi tổ hợp con.
- **MFI (maximal frequent itemset list):** danh sách các tập phổ biến tối đại được FP-Max duy trì trong quá trình khai thác [Grahne and Zhu \(2003\)](#).

4 Nền tảng lý thuyết

4.1 Bài toán Frequent Itemset Mining

Gọi $I = \{i_1, i_2, \dots, i_m\}$ là tập tất cả item. Một cơ sở dữ liệu giao dịch được ký hiệu là $D = \{T_1, T_2, \dots, T_n\}$, trong đó mỗi giao dịch $T_j \subseteq I$. Một itemset X là một tập con của I . Ta nói giao dịch T_j chứa X nếu $X \subseteq T_j$.

Độ hỗ trợ tuyệt đối của X trên D là số giao dịch chứa X :

$$\text{sup}(X) = |\{T_j \in D \mid X \subseteq T_j\}|.$$

Độ hỗ trợ tương đối là $\text{sup}_{rel}(X) = \text{sup}(X)/|D|$. Với ngưỡng hỗ trợ tối thiểu θ , itemset X được gọi là phổ biến nếu $\text{sup}_{rel}(X) \geq \theta$, hay tương đương $\text{sup}(X) \geq \lceil \theta |D| \rceil$.

Bên cạnh tập phổ biến thông thường, hai biến thể quan trọng là tập đóng và tập tối đại. Một itemset phổ biến X là *closed itemset* nếu không tồn tại itemset phổ biến Y sao cho $X \subset Y$ và $\text{sup}(X) = \text{sup}(Y)$. Một itemset phổ biến X là *maximal itemset* nếu không tồn tại itemset phổ biến Y sao cho $X \subset Y$. Do đó mọi maximal itemset đều là closed itemset, nhưng không phải mọi closed itemset đều là maximal itemset. Tập maximal là biểu diễn cô đọng nhất của không gian phổ biến: biết toàn bộ maximal itemset là đủ để xác định một itemset bất kỳ có phổ biến hay không (nó phổ biến khi và chỉ khi là tập con của một maximal itemset), dù không khôi phục được chính xác support của từng tập con. Đây chính là đối tượng mà FP-Max khai thác.

Tính chất quan trọng nhất của FIM là tính phản đơn điệu của support, thường được gọi là tính chất Apriori [Agrawal et al. \(1993\)](#). Nếu $X \subseteq Y$ thì mọi giao dịch chứa Y cũng chứa X , nên:

$$\text{sup}(Y) \leq \text{sup}(X).$$

Hệ quả là nếu X không phổ biến thì mọi siêu tập của X cũng không phổ biến. Ngược lại, nếu Y phổ biến thì mọi tập con của Y cũng phổ biến. Tính chất này cho phép các thuật toán FIM tìm kiếm không gian tìm kiếm rất mạnh, và cũng là cơ sở để FP-Max tìm theo siêu tập như trình bày bên dưới.

4.2 Bộ máy nền: FP-Growth và FP-tree

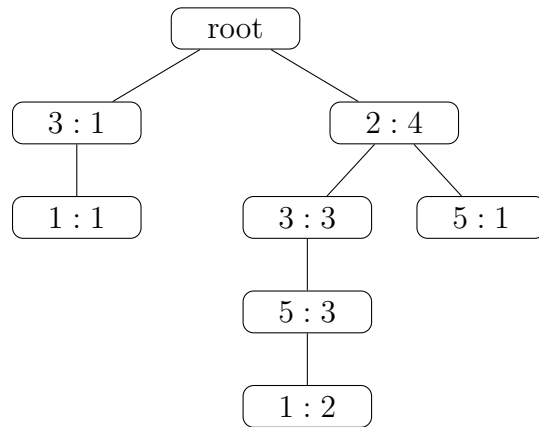
FP-Max không khai thác trực tiếp từ cơ sở dữ liệu mà tái sử dụng cấu trúc FP-tree cùng cơ chế cơ sở mẫu điều kiện của FP-Growth. Vì vậy phần này tóm tắt bộ máy nền đó trước khi đi vào FP-Max.

Apriori khai thác tính chất phản đơn điệu bằng cách sinh ứng viên theo từng mức độ dài [Agrawal and Srikant \(1994\)](#). Cách làm này dễ hiểu nhưng phải quét cơ sở dữ liệu nhiều lần và có thể sinh rất nhiều ứng viên trung gian. FP-Growth thay đổi hướng tiếp cận: thuật toán chỉ quét dữ liệu hai lần để xây dựng một cây nén gọi là FP-tree, sau đó khai thác mẫu bằng các cơ sở mẫu điều kiện mà không cần sinh ứng viên toàn cục [Han et al. \(2000\)](#).

Trực giác của FP-Growth là nhiều giao dịch có tiền tố giống nhau sau khi các item được sắp theo support giảm dần. Khi chèn các giao dịch đã sắp xếp vào cùng một cây, các tiền tố chung được gộp lại và đếm bằng biến *count* trên node. Header table lưu danh sách các node ứng với cùng một item, nhờ đó thuật toán có thể tìm nhanh các đường đi kết thúc tại item đang xét. Với mỗi item, thuật toán thu thập conditional pattern base, dựng conditional FP-tree, rồi đệ quy khai thác các mẫu có hậu tố là item đó.

Một node trong FP-tree gồm bốn thành phần: item đang đại diện, biến đếm số giao dịch đi qua node, con trỏ tới node cha, và tập các node con. Root không chứa item thật. Header table ánh xạ mỗi item tới danh sách các node mang item đó trong cây.

Ví dụ với cơ sở dữ liệu toy ở Mục 5 và $\theta = 0.6$, các item phổ biến là 2, 3, 5, 1 với support lần lượt 4, 4, 4, 3. Sau khi loại item không phổ biến và sắp item theo support giảm dần, FP-tree có dạng khái niệm trong Hình 1.



Hình 1: FP-tree minh hoạ cho CSDL toy ở Mục 5 với minsup = 0.6.

Cây trên nén năm giao dịch thành sáu node item. Hai giao dịch $[2, 3, 5, 1]$ trùng tiền tố hoàn toàn nên được biểu diễn bằng một đường đi có count tăng lên. Đây là nguồn tiết kiệm bộ nhớ chính của FP-tree, và FP-Max được hưởng lợi trực tiếp từ tính chất nén này. Bộ máy FP-Growth khai thác từng item: thu thập conditional pattern base, tĩa các item không đủ support, dựng conditional FP-tree và đệ quy; khi cây điều kiện chỉ còn một đường đi thì sinh trực tiếp các tổ hợp con thay vì dựng thêm cây.

4.3 Ý tưởng và thuật toán FP-Max

FP-Max khai thác tập phổ biến tối đại thay vì toàn bộ tập phổ biến [Grahne and Zhu \(2003\)](#). Điểm khác biệt cốt lõi so với FP-Growth là: thay vì xuất ra mọi itemset phổ biến gặp được, FP-Max chỉ giữ lại các itemset không có siêu tập phổ biến nào, và dùng chính các tối đại đã tìm được để *tĩa* những nhánh chắc chắn không sinh thêm tối đại mới. Nhờ đó FP-Max tránh được việc liệt kê số lượng tập phổ biến có thể lớn theo cấp số mũ trên dữ liệu dày.

FP-Max sử dụng hai cấu trúc đặc trưng:

- **Danh sách MFI** (maximal frequent itemset list): tập các tối đại đã tìm được. Khi có một candidate mới, nếu nó là tập con của một MFI sẵn có thì bỏ qua; ngược lại, mọi MFI cũ là tập con nghiêm ngặt của candidate bị loại, rồi thêm candidate vào. Bất biến của danh sách là không có hai phần tử nào lồng nhau.
- **Tĩa theo siêu tập** (superset pruning): tại một head hiện tại, gọi tail là tập các item còn đủ support trong conditional pattern base. Nếu $head \cup tail$ đã là tập con của một MFI có sẵn

thì toàn bộ nhánh đệ quy bắt nguồn từ head không thể sinh ra tối đại mới (mọi tối đại của nhánh đều nằm trong $head \cup tail$, do đó nằm trong MFI đó), nên nhánh bị cắt mà không cần dựng conditional FP-tree.

Để phép tĩa phát huy tác dụng sớm, FP-Max xét các item trong header theo thứ tự support tăng dần. Item hiếm được xử lý trước thường sinh ra các tối đại "sâu và dài", giúp danh sách MFI lớn nhanh và các item phổ biến hơn (xử lý sau) dễ rơi vào trường hợp bị tĩa. Khi cây điều kiện co lại thành một đường đi, toàn bộ đường đi hợp với head tạo thành đúng một candidate tối đại, không cần liệt kê $2^k - 1$ tập con như FP-Growth.

Algorithm 1 FP-Max

Require: FP-tree, head hiện tại, danh sách MFI

Ensure: Danh sách maximal frequent itemsets

```

1: if FP-tree chỉ có một đường đi then                                ▷ trường hợp tốt nhất: một candidate duy nhất
2:    $candidate \leftarrow head \cup$  các item trên đường đi
3:   Chèn  $candidate$  vào MFI nếu nó không là tập con của MFI hiện có
4:   Xoá các MFI cũ là tập con nghiêm ngặt của  $candidate$ 
5:   return
6: end if
7: Sắp các item trong header theo support tăng dần                                ▷ tĩa sớm
8: for mỗi item  $a$  do
9:    $head \leftarrow head \cup \{a\}$ 
10:  Tạo conditional pattern base và  $tail =$  các item còn phổ biến của  $a$ 
11:  if  $head \cup tail$  không là tập con của MFI nào then                                ▷ superset pruning
12:    Dựng conditional FP-tree và đệ quy với head mới
13:  end if                                ▷ ngược lại: cắt cả nhánh, không dựng cây điều kiện
14:  Bỏ  $a$  khỏi head
15: end for

```

Điểm mấu chốt là FP-Max không bao giờ vật chất hoá toàn bộ tập phổ biến. Mỗi tối đại được phát hiện hoặc ở nhánh single-path, hoặc khi một nhánh đệ quy kết thúc; còn các nhánh không thể đóng góp tối đại mới bị loại nhờ phép kiểm tra siêu tập. Support cuối cùng của mỗi MFI được tính lại bằng một lần đối chiếu với cơ sở dữ liệu, vì trong quá trình tĩa thuật toán chỉ quan tâm tới tính phổ biến chứ chưa cần giá trị support chính xác.

4.4 Phân tích độ phức tạp

Độ phức tạp thời gian. Chi phí gồm hai giai đoạn: xây FP-tree và khai thác đệ quy. Giai đoạn xây cây cần hai lần quét CSDL — một để đếm support, một để chèn giao dịch đã sắp — với chi

phí $\mathcal{O}(L \log \bar{\ell})$ do mỗi giao dịch phải sắp lại theo support (L là tổng số item trên toàn bộ giao dịch, $\bar{\ell}$ là độ dài giao dịch trung bình sau tĩa). Giai đoạn khai thác phụ thuộc số tập tối đại M và mức độ phép tĩa cắt được nhánh.

- **Tốt nhất (single-path):** cây co thành một đường, sinh trực tiếp đúng một tối đại mà không cần dựng conditional tree hay liệt kê tổ hợp con. Khai thác $\mathcal{O}(\bar{\ell})$, tổng $\mathcal{O}(L \log \bar{\ell})$. Đây là điểm vượt trội rõ rệt so với FP-Growth: trên cùng đường đi dài k , FP-Growth xuất $2^k - 1$ tập phổ biến còn FP-Max chỉ xuất một.
- **Trung bình:** phép tĩa theo siêu tập cắt phần lớn các nhánh không sinh tối đại mới, nên số conditional tree thực sự dựng nhỏ hơn nhiều số tập phổ biến. Chi phí xấp xỉ $\mathcal{O}(L \log \bar{\ell} + M \cdot c)$, trong đó c gộp chi phí trung bình dựng/duyet một cây điều kiện và chi phí kiểm tra/cập nhật danh sách MFI (mỗi thao tác $\mathcal{O}(|MFI| \cdot \bar{\ell})$ với danh sách MFI tuyến tính).
- **Xấu nhất:** mọi tập phổ biến đều là tối đại (một phản xích, không tập phổ biến nào chứa tập phổ biến khác), phép tĩa không cắt được gì và M có thể đạt $\binom{m}{m/2} \approx 2^m / \sqrt{m}$. Chi phí khai thác bùng nổ mũ $\mathcal{O}(2^m)$, chi phối toàn bộ thời gian chạy.

Bảng 3: Độ phức tạp thời gian của FP-Max theo từng trường hợp.

Trường hợp	Xây cây	Khai thác
Tốt nhất	$\mathcal{O}(L \log \bar{\ell})$	$\mathcal{O}(\bar{\ell})$
Trung bình	$\mathcal{O}(L \log \bar{\ell})$	$\mathcal{O}(M \cdot c)$
Xấu nhất	$\mathcal{O}(L \log \bar{\ell})$	$\mathcal{O}(2^m)$

Kết quả này nhất quán với thực nghiệm: thời gian chạy tăng theo số tập tối đại đầu ra (Mục 7), và bước nhảy mạnh trong thí nghiệm độ dài giao dịch (Bảng 14) xuất hiện đúng khi cấu trúc dữ liệu đầy M tăng nhanh, phù hợp với cận mũ ở trường hợp xấu.

Độ phức tạp không gian. Bộ nhớ FP-Max gồm FP-tree cùng header table, các conditional FP-tree sinh trong đệ quy, danh sách MFI, và tập kết quả. Gọi N là số node thực tế của FP-tree, M số tập tối đại.

- **Tốt nhất (single-path):** mọi giao dịch chung tiền tố, cây co thành một đường dài $\mathcal{O}(\bar{\ell})$, không sinh conditional tree và danh sách MFI chỉ có một phần tử. Bộ nhớ $\mathcal{O}(\bar{\ell})$, độc lập với n .

- **Trung bình:** nén tiền tố tốt nên $N \ll L$; các conditional tree dọc một nhánh được giải phóng khi nhánh kết thúc, độ sâu đệ quy $\mathcal{O}(\bar{\ell})$; danh sách MFI chiếm $\mathcal{O}(M \cdot \bar{\ell})$. Bộ nhớ làm việc xấp xỉ $\mathcal{O}(N + M\bar{\ell})$, khớp với Bảng 13: peak RSS chỉ vượt sèn vài chục MB.
- **Xấu nhất:** không có tiền tố chung nên $N = \mathcal{O}(L)$; nếu mọi tập phổ biến đều tối đại thì $M = \mathcal{O}(2^m/\sqrt{m})$, danh sách MFI và output chiếm ưu thế. Tổng $\mathcal{O}(L + M\bar{\ell})$.

Bảng 4: Độ phức tạp không gian của FP-Max theo từng trường hợp.

Trường hợp	Cấu trúc	MFI + output	Tổng
Tốt nhất	$\mathcal{O}(\bar{\ell})$	$\mathcal{O}(\bar{\ell})$	$\mathcal{O}(\bar{\ell})$
Trung bình	$\mathcal{O}(N)$, $N \ll L$	$\mathcal{O}(M\bar{\ell})$	$\mathcal{O}(N + M\bar{\ell})$
Xấu nhất	$\mathcal{O}(L)$	$\mathcal{O}(M\bar{\ell})$	$\mathcal{O}(L + M\bar{\ell})$

Cận trên ở đây là *đỉnh thường trú* (`sys.maxrss`), khác với tổng `alloc_bytes` (Bảng 10) vốn cộng dồn cả các `Vector` tạm bị GC thu hồi — nên tổng cấp phát thường lớn hơn đỉnh thường trú nhiều lần.

Các yếu tố ảnh hưởng chính. Hiệu năng FP-Max phụ thuộc vào: (i) số tập tối đại M , vốn tăng nhanh khi minsup giảm hoặc dữ liệu dày; (ii) độ dài giao dịch trung bình $\bar{\ell}$, vì giao dịch dài làm cây sâu hơn và tăng số item đồng xuất hiện; (iii) mức độ chia sẻ tiền tố, quyết định độ nén của FP-tree. So với FP-Growth, FP-Max ít nhạy với số tập *phổ biến* hơn vì phép tĩa và biểu diễn tối đại cô đọng, nhưng vẫn nhạy với M trong trường hợp xấu.

4.5 Vị trí lịch sử

Apriori đặt nền móng cho FIM bằng tính chất phản đơn điệu, nhưng bị giới hạn bởi số lần quét dữ liệu và số lượng ứng viên lớn [Agrawal and Srikant \(1994\)](#). FP-Growth giải quyết hạn chế này bằng FP-tree và khai thác mẫu không sinh ứng viên [Han et al. \(2000\)](#), song trên dữ liệu dày nó vẫn phải xuất ra số tập phổ biến lớn theo cấp số mũ. FP-Max (trong họ FPgrowth*/FPmax* của Grahne và Zhu) [Grahne and Zhu \(2003\)](#) khắc phục đúng điểm này: tái dùng FP-tree nhưng chỉ khai thác biểu diễn tối đại, dùng danh sách MFI và phép tĩa theo siêu tập để cắt không gian tìm kiếm. Đổi lại, FP-Max chỉ giữ vùng biên của không gian phổ biến nên không khôi phục được support của mọi tập con nếu không quét lại dữ liệu — đây là lý do phần ứng dụng sinh luật kết hợp (Mục 8) vẫn

cần bộ máy khai thác đầy đủ tập phổ biến. Song song với hướng dạng cây, các thuật toán dạng dọc như Eclat dùng tidset/diffset [Zaki \(2000\)](#), và các thư viện thực nghiệm như SPMF cung cấp nhiều cài đặt tham chiếu [Fournier-Viger et al. \(2014\)](#), trong đó có FPMax dùng làm chuẩn đối chiếu ở Mục 7.

5 Ví dụ minh họa

5.1 Ví dụ 1: CSDL cơ sở

Xét cơ sở dữ liệu D gồm 5 giao dịch trong Bảng 5. Nhóm chọn minsup tương đối $\theta = 0.6$, do đó minsup tuyệt đối là $\lceil 0.6 \times 5 \rceil = 3$.

Bảng 5: CSDL toy dùng để minh họa FP-Max.

Giao dịch	Item
T_1	$\{1, 3, 4\}$
T_2	$\{2, 3, 5\}$
T_3	$\{1, 2, 3, 5\}$
T_4	$\{2, 5\}$
T_5	$\{1, 2, 3, 5\}$

Bước đầu tiên là đếm support của từng item. Kết quả là $\text{sup}(1) = 3$, $\text{sup}(2) = 4$, $\text{sup}(3) = 4$, $\text{sup}(4) = 1$, $\text{sup}(5) = 4$. Vì minsup tuyệt đối bằng 3, item 4 bị loại. Để dựng FP-tree, các item còn lại trong mỗi giao dịch được sắp theo support giảm dần, nếu bằng nhau thì theo mã item tăng dần: $2 \prec 3 \prec 5 \prec 1$.

Bảng 6: Các giao dịch sau bước tảo và sắp thứ tự để dựng FP-tree.

Giao dịch	Sau khi loại item không phổ biến	Sau khi sắp theo thứ tự FP-tree
T_1	$\{1, 3\}$	$[3, 1]$
T_2	$\{2, 3, 5\}$	$[2, 3, 5]$
T_3	$\{1, 2, 3, 5\}$	$[2, 3, 5, 1]$
T_4	$\{2, 5\}$	$[2, 5]$
T_5	$\{1, 2, 3, 5\}$	$[2, 3, 5, 1]$

Chèn tuần tự các giao dịch trong Bảng 6 tạo ra FP-tree đã minh họa ở Hình 1. Khác với FP-Growth, FP-Max duyệt header table theo thứ tự support *tăng dần*, tức $1 \prec 2 \prec 3 \prec 5$ (item 1 có support

3, ba item còn lại có support 4 nên xếp theo mã item). Bảng 7 ghi lại từng bước: head đang xét, conditional pattern base, tail (các item còn đủ support trong base), candidate tối đại sinh ra, và trạng thái danh sách MFI sau bước đó.

Bảng 7: Vòng đệ quy FP-Max trên ví dụ cơ sở. Mỗi candidate được hợp nhất vào danh sách MFI theo quy tắc loại tập con nghiêm ngặt.

Head	Conditional pattern base	Tail	Candidate	MFI sau bước
1	(3) : 1, (2, 3, 5) : 2	{3}	{1, 3}	{1, 3}
2	$\emptyset$	$\emptyset$	{2}	{1, 3}, {2}
3	(2) : 3	{2}	{2, 3}	{1, 3}, {2, 3}
5	(2, 3) : 3, (2) : 1	{2, 3}	{2, 3, 5}	{1, 3}, {2, 3, 5}

Diễn giải các bước trong Bảng 7. Khi xét head {1}, conditional pattern base gồm đường (3) với count 1 và đường (2, 3, 5) với count 2; chỉ item 3 đạt support 3 nên tail là {3}. Cây điều kiện co thành một đường, sinh candidate {1, 3} và khởi tạo MFI. Head {2} có base rỗng (node 2 nằm ngay dưới root) nên tạm thời cho candidate {2}. Đến head {3}, base (2) : 3 cho tail {2} và candidate {2, 3}; vì {2} là tập con nghiêm ngặt của {2, 3} nên {2} bị loại khỏi MFI. Cuối cùng head {5} có base (2, 3) : 3 và (2) : 1; tail {2, 3}, cây điều kiện single-path cho candidate {2, 3, 5}, làm {2, 3} bị loại. Danh sách MFI hội tụ về hai phần tử.

Sau khi tính lại support trên D , tập kết quả của FP-Max là:

$$\{1, 3\} : 3, \quad \{2, 3, 5\} : 3.$$

Để kiểm tra chéo thủ công, ta liệt kê toàn bộ tập phổ biến từ các item {1, 2, 3, 5}: các đơn item {1} : 3, {2} : 4, {3} : 4, {5} : 4; các cặp {1, 3} : 3, {2, 3} : 3, {2, 5} : 4, {3, 5} : 3; và bộ ba {2, 3, 5} : 3 (xuất hiện trong T_2, T_3, T_5). Trong chín tập phổ biến này, chỉ {1, 3} và {2, 3, 5} là không có siêu tập phổ biến: mọi tập còn lại đều là tập con của một trong hai. Kết quả thủ công trùng khớp với đầu ra của FP-Max, và cũng đúng với cài đặt FPMMax tham chiếu trong SPMF.

5.2 Ví dụ 2: Tỉa theo siêu tập

Trường hợp đặc biệt quan trọng của FP-Max là phép tỉa theo siêu tập: khi $head \cup tail$ đã nằm trong một MFI có sẵn, cả nhánh đệ quy bị cắt mà không cần dựng conditional FP-tree. Ví dụ này được thiết kế để phép tỉa thực sự kích hoạt.

Xét CSDL trong Bảng 8 với $\theta = 0.5$, minsup tuyệt đối bằng 3.

Bảng 8: CSDL thiết kế để kích hoạt phép tĩa theo siêu tập trong FP-Max.

Giao dịch	Item
T_1	$\{a, b, c\}$
T_2	$\{a, b, c\}$
T_3	$\{a, b, c\}$
T_4	$\{a, b, d\}$
T_5	$\{a, b, d\}$
T_6	$\{a, b, d\}$

Support của các item là $\text{sup}(a) = 6$, $\text{sup}(b) = 6$, $\text{sup}(c) = 3$, $\text{sup}(d) = 3$; tất cả đều đạt ngưỡng.

Thứ tự dựng cây theo support giảm dần là a, b, c, d , cho FP-tree:

$$\text{root} \rightarrow a : 6 \rightarrow b : 6 \rightarrow \{c : 3, d : 3\}.$$

Vì node b có hai con, cây không phải single-path nên thuật toán đi vào vòng đệ quy theo từng item.

Header được duyệt theo support tăng dần: c, d, a, b (hai item hiếm c, d trước, hai item phổ biến a, b sau). Bảng 9 ghi lại diễn biến.

Bảng 9: Vòng đệ quy FP-Max trên CSDL Bảng 8. Hai item cuối bị tĩa nhờ kiểm tra siêu tập.

Head	Conditional pattern base	$\text{head} \cup \text{tail}$	Hành động	MFI sau bước
c	$(a, b) : 3$	$\{a, b, c\}$	dựng cây, candidate $\{a, b, c\}$	$\{a, b, c\}$
d	$(a, b) : 3$	$\{a, b, d\}$	dựng cây, candidate $\{a, b, d\}$	$\{a, b, c\}, \{a, b, d\}$
a	$\emptyset$	$\{a\}$	$\{a\} \subseteq \{a, b, c\} \Rightarrow$ tĩa	$\{a, b, c\}, \{a, b, d\}$
b	$(a) : 6$	$\{a, b\}$	$\{a, b\} \subseteq \{a, b, c\} \Rightarrow$ tĩa	$\{a, b, c\}, \{a, b, d\}$

Hai item hiếm c và d được xử lý trước, mỗi item có conditional pattern base $(a, b) : 3$ nên sinh hai tối đại $\{a, b, c\}$ và $\{a, b, d\}$. Khi đến item b , conditional pattern base là $(a) : 6$ cho tail $\{a\}$; vì $\text{head} \cup \text{tail} = \{a, b\}$ đã là tập con của MFI $\{a, b, c\}$, thuật toán cắt cả nhánh và không dựng conditional FP-tree cho b — đây chính là chỗ tĩa tiết kiệm công sức. Item a tương tự bị tĩa với $\text{head} \cup \text{tail} = \{a\}$. Tập kết quả là $\{a, b, c\} : 3$ và $\{a, b, d\} : 3$.

Trường hợp này làm rõ điểm khác biệt cốt lõi giữa FP-Max và FP-Growth. FP-Growth buộc phải xuất ra mọi tập phổ biến — ở đây gồm $\{a\}, \{b\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}, \dots$ với tổng cộng mười tập — trong khi FP-Max chỉ giữ hai tối đại và bỏ qua phần lớn không gian nhờ phép tĩa. Cực điểm

của hiệu ứng này là khi FP-tree co thành một đường đi dài k : FP-Growth sinh $2^k - 1$ tập phổ biến, còn FP-Max nhận diện single-path và trả về đúng một tối đại. Đây là lý do FP-Max đặc biệt hiệu quả trên dữ liệu dày, nơi số tập phổ biến lớn theo cấp số mũ nhưng số tập tối đại nhỏ hơn nhiều.

6 Cài đặt

6.1 Môi trường và tổ chức mã nguồn

Cài đặt chính của nhóm sử dụng Julia theo đúng ưu tiên của đề bài. Mã nguồn được tổ chức như một package tên `FrequentItemsetMining`, trong đó file `src/FrequentItemsetMining.jl` là module entry và include các file con. Bố cục chính của repo như sau:

- `src/structures.jl`: định nghĩa `Transaction`, `DataLoader`, `FPNode`, `FPNodeOpt`.
- `src/utils.jl`: tiện ích chung, ví dụ hàm `combinations` sinh tổ hợp dùng trong khai thác và sinh luật.
- `src/data_loader.jl`: đọc dữ liệu giao dịch, hỗ trợ dòng item cách nhau bằng khoảng trắng theo định dạng SPMF và dòng có dấu phẩy.
- `src/algorithm/fp_growth.jl`: cài đặt FP-Growth cơ bản.
- `src/algorithm/fp_growth_opt.jl`: cài đặt FP-Growth tối ưu.
- `src/algorithm/fp_max.jl`: cài đặt FP-Max.
- `src/association_rules.jl`: định nghĩa `AssociationRule` và `generate_rules` để sinh luật kết hợp từ tập phổ biến.
- `src/io.jl`: xuất itemsets theo dạng `item item ... #SUP: count`.
- `main.jl`: giao diện dòng lệnh.
- `test/`: bộ unit test correctness và benchmark smoke test.
- `experiments/`: script đối chiếu SPMF, sinh CSV và sinh hình cho Mục 7.

Trước khi chuyển sang Julia, nhóm có một bản phác thảo (prototype) bằng Python đặt trong thư mục `python/`. Bản này chỉ dùng để kiểm chứng ý tưởng ban đầu của FP-Growth và FP-Max; toàn bộ kết quả, thực nghiệm và đánh giá trình bày trong báo cáo đều dựa trên cài đặt Julia.

Lệnh chạy chương trình có dạng:

```
1 julia --project=. main.jl <input_path> <minsup> [fp-growth|fp-growth-opt|fp-max|
  rules] [output_path]
```

Listing 1: Cách chạy chương trình từ dòng lệnh.

Ví dụ, để khai thác tập phổ biến trên CSDL toy với ngưỡng 0.6:

```
1 julia --project=. main.jl data/toy/test_1.txt 0.6 fp-growth-opt output.txt
```

Listing 2: Ví dụ chạy FP-Growth tối ưu.

Ngoài ba thuật toán khai thác itemset, CLI còn có chế độ `rules` để sinh trực tiếp luật kết hợp từ tập phổ biến (dùng FP-Growth tối ưu), lọc theo lift > 1 và sắp theo lift giảm dần; ngưỡng confidence đặt qua biến môi trường `MINCONF` (mặc định 0.2). Với dữ liệu có item chứa khoảng trắng trong tên như Groceries, đặt `SEP=comma` để parser chỉ tách theo dấu phẩy:

```
1 SEP=comma julia --project=. main.jl data/groceries.txt 0.01 rules rules.csv
```

Listing 3: Sinh luật kết hợp trên Groceries qua CLI.

Lệnh này tái tạo đúng kết quả của phần ứng dụng (Mục 8): 333 frequent itemsets và 231 luật thỏa `minconf` với lift lớn hơn 1.

6.2 Cài đặt FP-Growth cơ bản

Bản cơ bản dùng node lưu item dạng `String`. Mỗi node có `count`, con trỏ `parent` và tập `children`. Khi chèn item vào một node cha, chương trình duyệt các con hiện có để tìm child cùng item; nếu có thì tăng `count`, nếu không thì tạo node mới. Sau khi xây cây, hàm `rebuild_nodes_table!` duyệt cây để dựng header table.

Quy trình `fit!` của `FPGrowth` gồm bốn bước. Thứ nhất, gom toàn bộ giao dịch và tính `minsup` tuyệt đối bằng $\lceil \theta n \rceil$. Thứ hai, đếm support từng item và loại item không đủ ngưỡng. Thứ ba, trong mỗi giao dịch, giữ lại item phổ biến và sắp theo (`-support`, `item`) để kết quả có thứ tự ổn định. Cuối cùng, chèn các giao dịch vào FP-tree.

Hàm `get_frequent_itemsets` khai thác từng item bằng `mine_roads`. Với mỗi node của item đang xét, hàm đi ngược theo con trỏ cha để lấy đường tiền tố. Các đường tiền tố tạo thành conditional pattern base. Bản cơ bản sau đó đếm các item hợp lệ trong base, sinh các tổ hợp con của chúng và tính support bằng cách kiểm tra đường nào chứa tổ hợp đó. Cách này dễ kiểm chứng và phù hợp làm baseline, nhưng có thể tốn kém khi conditional base rộng vì phải enumerate nhiều tổ hợp.

6.3 Cài đặt FP-Growth tối ưu

Bản `FPGrowthOpt` giữ cùng API với bản cơ bản nhưng thay đổi cấu trúc dữ liệu và cách khai thác. Các item được mã hoá từ `String` sang `Int`. Mỗi node tối ưu lưu `item::Int`, `count::Int`, `parent`, và `children::Dict{Int, FPNODEOpt}`. Nhờ vậy thao tác tìm child khi chèn path có chi phí trung bình $O(1)$ thay vì duyệt tuyến tính qua tập con.

Các kỹ thuật tối ưu chính gồm:

- **Integer encoding**: khai thác trên mã số nguyên và chỉ giải mã về chuỗi ở bước cuối.
- **Header table typed**: header ánh xạ `Int` sang danh sách node, giảm overhead so với item chuỗi.
- **Recursive conditional FP-tree**: khai thác đúng hướng FP-Growth chuẩn, tránh enumerate tổ hợp trên conditional base rộng như bản cơ bản.
- **Single-path shortcut**: nếu cây điều kiện chỉ có một nhánh, sinh trực tiếp mọi tổ hợp con của nhánh đó.
- **Tỉa sớm**: trong mỗi conditional pattern base, item có support nhỏ hơn minsup tuyệt đối bị loại trước khi dựng cây điều kiện.
- **Type-stable hot loop**: các trường dữ liệu được khai báo kiểu cụ thể và một số vòng lặp dùng `@inbounds` khi truy cập vector.

Output của bản tối ưu vẫn là `Dict{Tuple{Vararg{String}}, Int}`, giống bản cơ bản. Điều này giúp unit test có thể so sánh trực tiếp:

```
get_frequent_itemsets(FPGrowthOpt) == get_frequent_itemsets(FPGrowth)
```

6.4 Tránh performance anti-patterns

Ở bản tối ưu, nhóm không chỉ thêm các macro hiệu năng vào bản cơ bản mà tổ chức lại dữ liệu để trình biên dịch Julia có đủ thông tin về kiểu trong các đoạn chạy nhiều nhất. Trạng thái của thuật toán được gom trong `FPGrowthOpt`: ngưỡng support, bảng mã hoá item, root của FP-tree và header table đều là các trường có kiểu cụ thể. Vì vậy quá trình dựng cây và khai thác không phụ thuộc vào biến toàn cục thay đổi được, tránh tình huống Julia phải suy luận kiểu lại qua từng lần truy cập.

Các node trên FP-tree cũng được thiết kế theo hướng này. `FPNodeOpt` lưu item dưới dạng `Int`, count dưới dạng `Int`, và tập con bằng `Dict{Int, FPNodeOpt}`. Header table tương ứng có kiểu `Dict{Int, Vector{FPNodeOpt}}`. Những cấu trúc này giúp thao tác chèn path và tính support đi qua các container có kiểu rõ ràng, thay vì làm việc với item chuỗi hoặc container abstract trong hot loop. Trường `parent::Union{Nothing, FPNodeOpt}` vẫn được giữ vì cần cho bước dựng conditional pattern base; đây là một union nhỏ và không làm thay đổi kiểu của node hay header table.

Các macro tối ưu được dùng có chọn lọc. `@inbounds` chỉ xuất hiện ở những vòng lặp mà biên truy cập đã được đảm bảo bởi chính cấu trúc vòng lặp, như duyệt path trong `insert_path!`, cộng support trong `_item_support`, và sinh tổ hợp trên single-path trong `_mine!`. Ngược lại, `@simd` không được dùng vì phần tốn thời gian của FP-Growth chủ yếu là duyệt cây và tra cứu `Dict`. Đây là dạng truy cập rời rạc, phụ thuộc con trỏ và nhánh điều kiện; ép vector hoá ở đây không phù hợp với bản chất của thuật toán.

Nhóm kiểm chứng tính ổn định kiểu bằng `@code_warntype` và `@inferred` trên các hàm nóng. Với `_item_support`, `@code_warntype` cho thấy mọi biến cục bộ và kiểu trả về đều cụ thể:

Arguments

```
nodes::Vector{FPNodeOpt}
```

Locals

```
s::Int64
```

```
nd::FPNodeOpt
```

Body::Int64

Không có biến nào bị suy ra kiểu `Any`; union duy nhất xuất hiện là trạng thái iterator chuẩn của vòng lặp `for` trong Julia, không phải bất ổn kiểu. Lệnh `@inferred FrequentItemsetMining._item_support(nodes)` chạy không lỗi và suy ra `Int64`, xác nhận hàm nóng này type-stable.

Tính ổn định kiểu của bộ máy FP-tree này được cả hai chiến lược khai thác maximal sử dụng: baseline naive-maximal chạy thẳng trên `FPGrowthOpt`, còn FP-Max dùng lại cùng cơ chế cây và cơ sở mẫu điều kiện. Hiệu quả tổng thể của tối ưu được đo ở Mục 6.5 và Mục 7.

6.5 Cài đặt FP-Max và baseline naive-maximal

Nhóm cài đặt khai thác tập tối đại theo hai chiến lược để có thể đo lường tác động của phép tĩa.

Baseline naive-maximal. Hàm `maximal_from_frequent` nhận đầu ra đầy đủ của `get_frequent_itemsets` (chạy trên `FPGrowthOpt`) rồi loại mọi itemset có một siêu tập nghiêm ngặt cũng phổ biến. Đây là cách trực tiếp nhất để thu tập tối đại, nhưng phải vật chất hoá toàn bộ tập phổ biến trước khi lọc, nên thừa hưởng đúng điểm bùng nổ của FP-Growth trên dữ liệu dày: ở minsup thấp, số tập phổ biến lớn theo cấp số mũ làm bước mine-all không khả thi.

FP-Max trực tiếp. FPMax dựng FP-tree ban đầu qua `FPGrowth` và lưu lại toàn bộ giao dịch dưới dạng `Set{String}` để tính support của các maximal itemset ở bước cuối. Hàm `get_maximal_itemsets` duy trì danh sách MFI dưới dạng vector các set. Khi có candidate mới, `insert_mfi!` bỏ qua candidate nếu nó là tập con của một MFI sẵn có; ngược lại, hàm xoá các MFI cũ là tập con nghiêm ngặt của candidate rồi thêm candidate mới, giữ bất biến không có hai MFI nào lồng nhau. Trong đệ quy, nếu cây là single-path thì hợp head hiện tại với toàn bộ item trên đường đi thành một candidate duy nhất. Nếu cây nhiều nhánh, thuật toán xét item trong header theo support tăng dần, dựng conditional pattern base, tìm tail còn phổ biến, và dùng phép kiểm tra $head \cup tail$ để tĩa: nếu $head \cup tail$ đã nằm trong một MFI thì cắt cả nhánh, không dựng conditional FP-tree. Nhờ đó FP-Max chạy được trong những cấu hình mà baseline naive-maximal bị bỏ qua vì bùng nổ bộ nhớ.

Đo lường tối ưu. Bảng 10 so sánh lượng cấp phát của hai chiến lược trên Chess ở ba ngưỡng minsup mà cả hai còn chạy được (cột `alloc_bytes` từ `timing.csv`). Lợi ích của phép tĩa lớn dần khi tỉ lệ giữa số tập phổ biến và số tập tối đại tăng: ở minsup 0.80, baseline cấp phát 836.98 MB còn FP-Max chỉ 34.26 MB — giảm khoảng 24.4 lần; ở minsup 0.85 tỉ lệ là 4.75 lần; tới minsup 0.90 hai chiến lược xấp xỉ nhau vì số tập phổ biến chưa nhiều hơn số tối đại bao nhiêu. Quan trọng hơn, tại các ngưỡng thấp hơn (0.75, 0.70) baseline bị bỏ qua hoàn toàn vì mine-all vượt bộ nhớ, trong khi FP-Max vẫn hoàn tất — đây mới là lợi ích quyết định của khai thác tối đại trực tiếp. Phân

tích thời gian chi tiết, gồm cả vùng dữ liệu thừa nơi chi phí cài đặt của FP-Max lấn át lợi ích tủa, được trình bày ở Mục 7.

Bảng 10: Bộ nhớ cấp phát của baseline naive-maximal và FP-Max trên Chess, đo bằng `alloc_bytes`.

minsup	naive-maximal (MB)	FP-Max (MB)	Tỉ lệ giảm
0.80	836.98	34.26	24.43
0.85	119.71	25.23	4.75
0.90	18.64	20.17	0.92

6.6 Kiểm thử tự động

Bộ kiểm thử trong `test/test_correctness.jl` gồm nhiều nhóm. Nhóm parser kiểm tra ba định dạng: 1, 2, 3, {1, 2, 3}, và 1 2 3. Nhóm trọng tâm kiểm tra FP-Max trên CSDL toy ở Mục 5, kỳ vọng đúng hai maximal itemsets {1, 3} : 3 và {2, 3, 5} : 3 — chính kết quả của ví dụ tay. Một nhóm riêng kiểm tra baseline `maximal_from_frequent` cho ra cùng hai tối đại đó, xác nhận hai chiến lược (naive-maximal và FP-Max trực tiếp) đồng nhất về kết quả. Nhóm còn lại kiểm tra bộ máy nền FP-Growth cơ bản (9 frequent itemsets trên CSDL toy) để bảo đảm cây và cơ sở mẫu điều kiện mà FP-Max dựa vào là đúng.

Ngoài ra, test đối chiếu FPGrowthOpt với FPGrowth trên `data/toy/test_1.txt`, `data/toy/test_2.txt` và `data/benchmark/chess.txt` ở nhiều giá trị minsup. Test benchmark trong `test/test_benchmark.jl` đo thời gian và số byte cấp phát trên `chess` với minsup 0.9, sau đó kiểm tra lại kết quả của bản tối ưu và bản cơ bản phải trùng nhau. Lệnh kiểm thử chuẩn là:

```
1 julia --project=. test/runtests.jl
```

Listing 4: Lệnh chạy bộ test.

6.7 Xử lý đầu vào và đầu ra

Input chuẩn của đồ án là mỗi giao dịch trên một dòng, item cách nhau bằng khoảng trắng theo định dạng SPMF. Parser cũng hỗ trợ dòng có dấu phẩy để thuận tiện cho dữ liệu giỏ hàng. Output của `write_itemsets` sắp các itemset theo độ dài và thứ tự từ điển, sau đó ghi mỗi dòng theo dạng:

```
1 item1 item2 ... #SUP: support_count
```

Listing 5: Định dạng output itemset.

Định dạng này tương thích với output của SPMF, giúp script thực nghiệm đọc kết quả của hai phía và so sánh từng itemset theo support.

7 Thực nghiệm và đánh giá

7.1 Thiết lập thực nghiệm

Thực nghiệm được thiết kế để kiểm tra sáu khía cạnh: tính đúng đắn so với SPMF FPMax, thời gian chạy theo minsup, số lượng tập tối đại theo minsup, bộ nhớ cấp phát, khả năng mở rộng theo số giao dịch, và ảnh hưởng của độ dài giao dịch trung bình. Kết quả được lưu trong các file CSV ở `experiments/results/`; hình minh hoạ được sinh vào `experiments/figures/`.

Các cài đặt được so sánh gồm:

- **base**: baseline naive-maximal của nhóm — khai thác toàn bộ tập phổ biến bằng `FPGrowthOpt` rồi lọc các tập tối đại.
- **opt**: FP-Max trực tiếp của nhóm — khai thác tối đại có danh sách MFI và phép tỉa theo siêu tập.
- **spmf**: cài đặt FPMax trong thư viện SPMF [Fournier-Viger et al. \(2014\)](#), dùng làm tham chiếu.

Baseline naive-maximal phải vật chất hoá toàn bộ tập phổ biến trước khi lọc, nên không chạy ở các cấu hình có nguy cơ bùng nổ bộ nhớ, đặc biệt trên dữ liệu dày hoặc minsup thấp. Các cấu hình này được đánh dấu `skip` trong CSV thay vì ép chạy đến khi tiến trình bị lỗi; riêng `accidents` luôn skip base. Đây cũng chính là điểm khác biệt then chốt: FP-Max trực tiếp vẫn hoàn tất trong những vùng mà baseline không khả thi. Với Julia, thời gian được đo sau một lần warm-up để giảm ảnh hưởng của JIT; lượng cấp phát dùng cột `alloc_bytes` từ `@timed`, còn peak memory thật được đo riêng bằng `Sys.maxrss` trong tiến trình độc lập (xem Mục Bộ nhớ). Với SPMF, thời gian và bộ nhớ lấy từ log của chương trình Java.

7.2 Tập dữ liệu benchmark

Bảng 11 tóm tắt các tập dữ liệu dùng trong thực nghiệm. Các tập dữ liệu này là benchmark phổ biến trong cộng đồng FIM, lấy từ kho dữ liệu FIMI [Goethals and Zaki \(2003\)](#). Các số liệu trong bảng được tính trực tiếp từ file trong thư mục `data/benchmark`.

Bảng 11: Các tập dữ liệu benchmark dùng trong Mục 7.

Dataset	#Trans.	#Items	AvgLen	Đặc điểm
Chess	3,196	75	37.00	Dày, giao dịch dài
Mushrooms	8,416	119	23.00	Dày, nhiều item đồng xuất hiện
Retail	88,162	16,470	10.31	Thưa, dữ liệu bán lẻ thực
T10I4D100K	100,000	870	10.10	Tổng hợp, thưa
Accidents	340,183	468	33.80	Rất lớn và dày

7.3 Tính đúng đắn

Kết quả trong `correctness.csv` cho thấy mọi cấu hình được kiểm tra đều khớp hoàn toàn với SPMF FPMax: `match_ratio = 1.0` và `support_mismatch = 0`, cho cả bản `base` lẫn `opt`. Bảng 12 liệt kê một số điểm đại diện, trong đó `#Ours` và `#SPMF` là số tập tối đại.

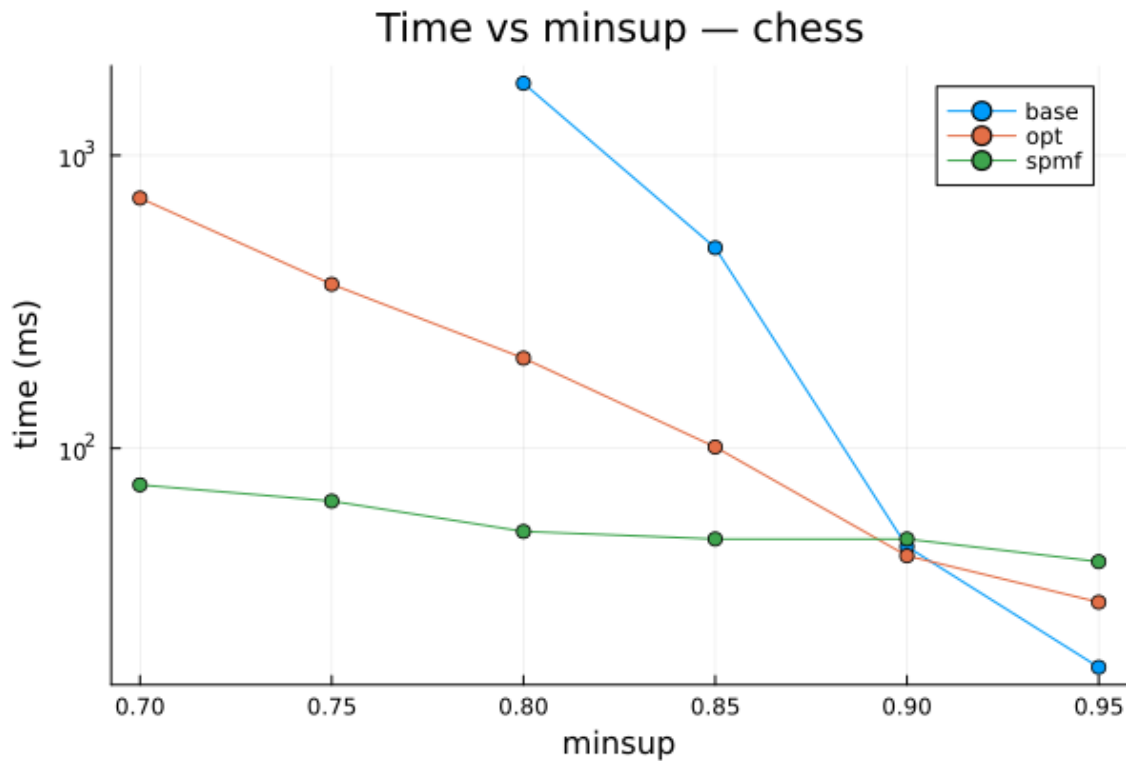
Bảng 12: Kết quả correctness đại diện khi so số tập tối đại với SPMF FPMax.

Dataset	minsup	Algo	#Ours	#SPMF	Match ratio
Chess	0.80	base/opt	226	226	1.0
Mushrooms	0.25	base/opt	110	110	1.0
Retail	0.01	base/opt	78	78	1.0
T10I4D100K	0.01	base/opt	370	370	1.0
Accidents	0.70	opt	23	23	1.0

Kết quả này xác nhận ba điểm. Thứ nhất, parser, cách tính minsup tuyệt đối và support của nhóm tương thích với SPMF trên dữ liệu benchmark. Thứ hai, cài đặt FP-Max trực tiếp cho đúng tập tối đại như cài đặt tham chiếu. Thứ ba, hai chiến lược `base` và `opt` của nhóm cho cùng kết quả, nên phép tĩa theo siêu tập không làm mất tối đại nào.

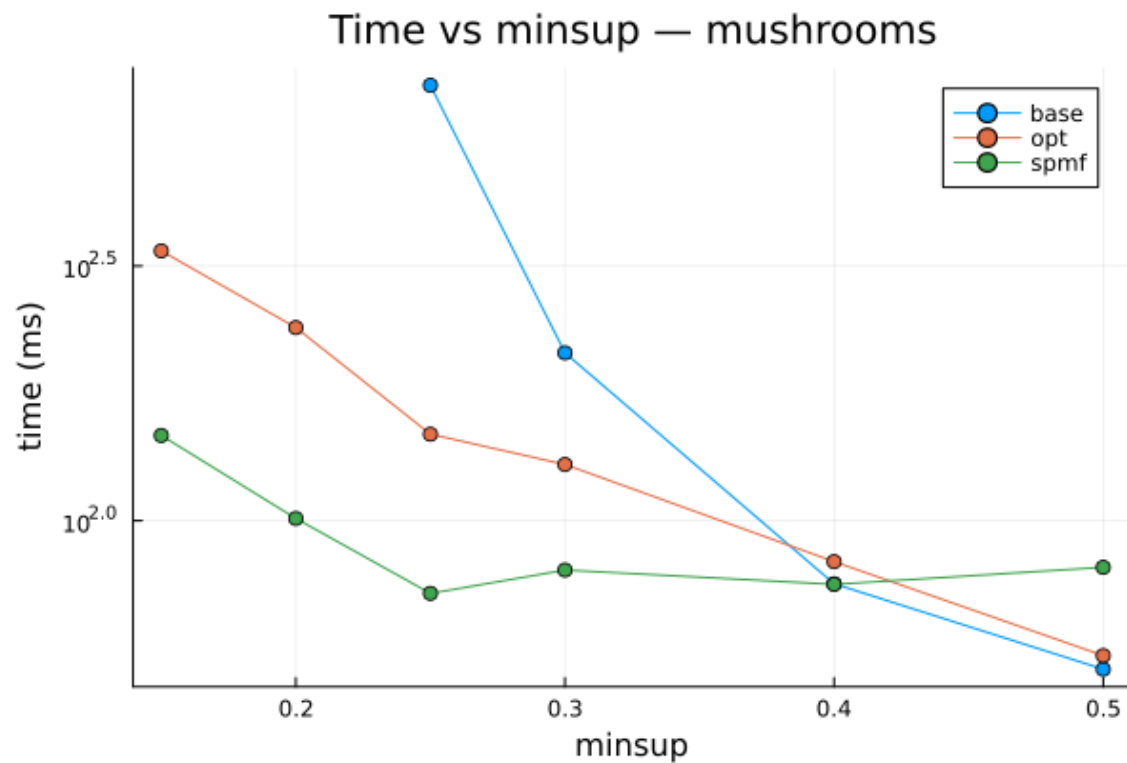
7.4 Thời gian chạy theo minsup

Hình 2 đến Hình 5 cho thấy thời gian chạy khi minsup giảm. Xu hướng chung là minsup càng thấp thì số tập tối đại càng tăng và thời gian chạy tăng theo. Điều này phù hợp với phân tích lý thuyết: khi ngưỡng thấp, nhiều item sống sót qua bước tĩa hơn, conditional tree rộng hơn và số tối đại lớn hơn.



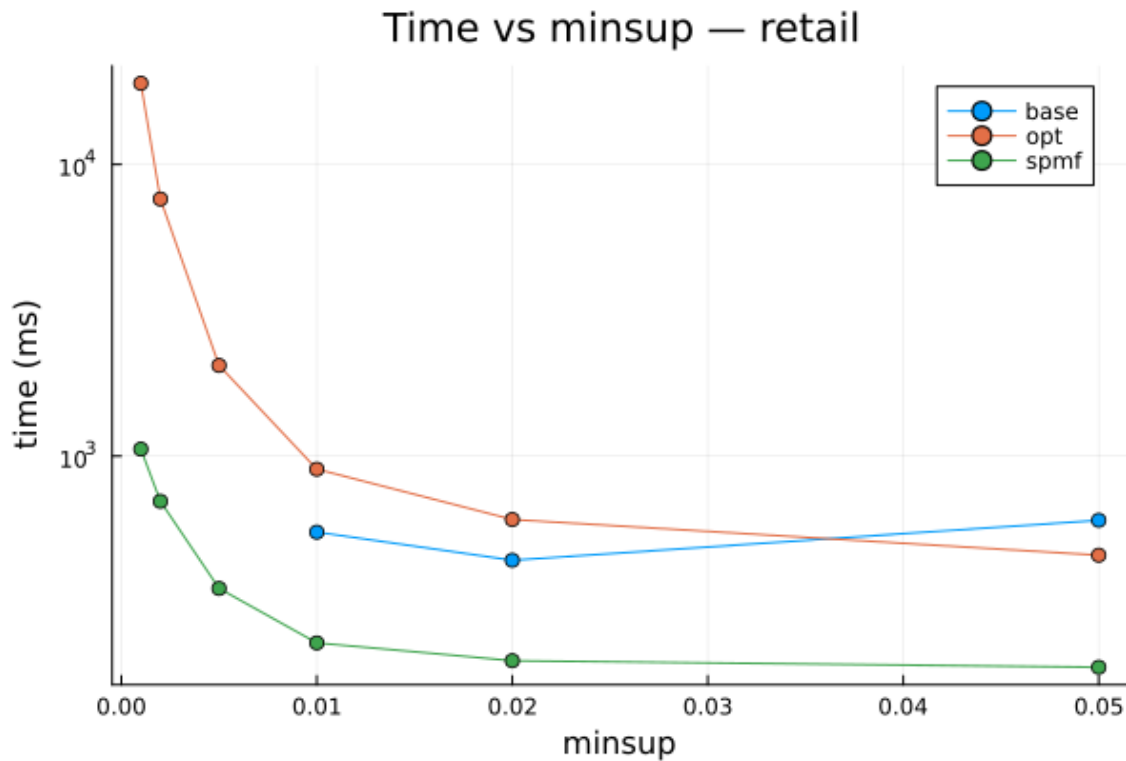
Hình 2: Thời gian chạy theo minsup trên Chess.

Trên Chess, FP-Max trực tiếp (**opt**) thể hiện lợi thế rõ ở vùng output lớn. Tại minsup 0.80, baseline naive-maximal mất 1762.92 ms trong khi **opt** chỉ mất 203.07 ms, nhanh hơn khoảng 8.68 lần; ở các ngưỡng thấp hơn (0.75, 0.70) baseline bị skip vì mine-all bùng nổ, còn **opt** vẫn chạy (714.94 ms ở minsup 0.70 với 891 tối đại). Khi minsup rất cao như 0.95, output chỉ còn 11 tối đại nên chi phí cố định chiếm ưu thế và baseline lại nhanh hơn đôi chút (17.88 ms so với 29.81 ms). So với SPMF, **opt** vẫn chậm hơn (203.07 ms so với 52 ms ở minsup 0.80), phản ánh khoảng cách giữa cài đặt học thuật và thư viện Java đã tối ưu lâu dài.



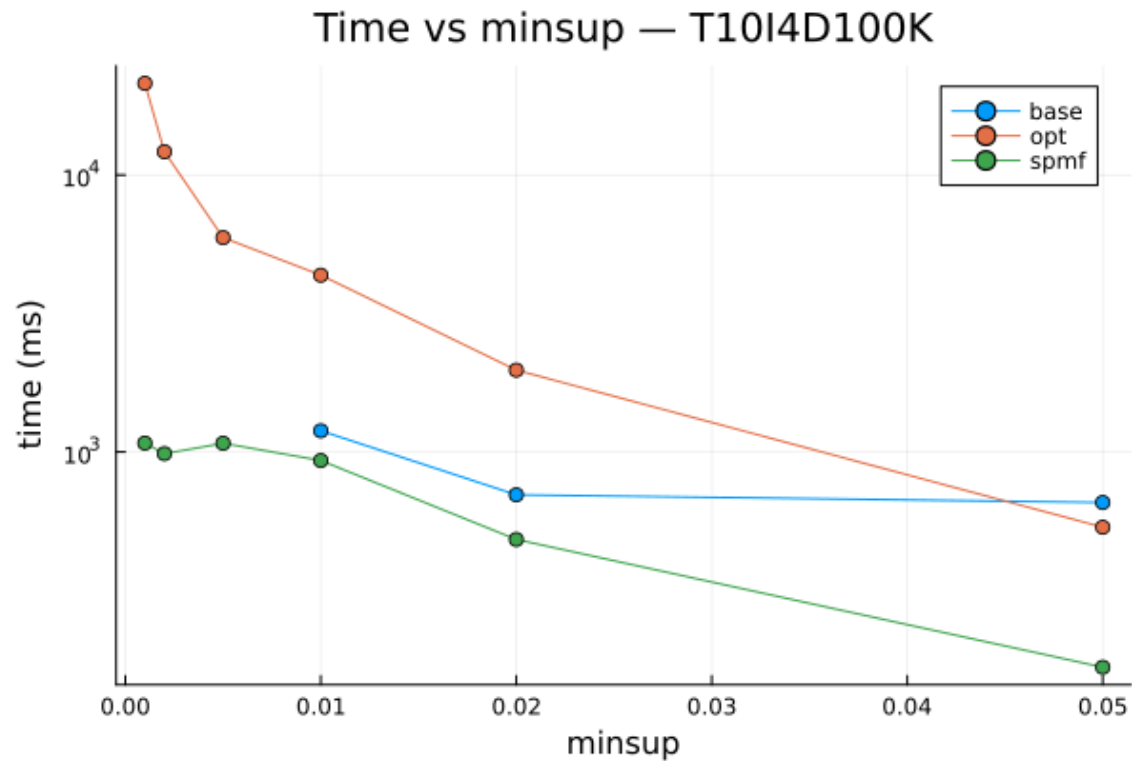
Hình 3: Thời gian chạy theo minsup trên Mushrooms.

Hình 3 thể hiện xu hướng tương tự trên Mushrooms. Tại minsup 0.25, **opt** nhanh hơn baseline khoảng 4.84 lần (147.81 ms so với 715.35 ms); ở minsup 0.30 tỉ lệ thu hẹp còn 1.66 lần. Ở các ngưỡng cao (0.4, 0.5) khi output nhỏ, hai bản xấp xỉ nhau và chi phí dựng cây cùng JIT chi phối.



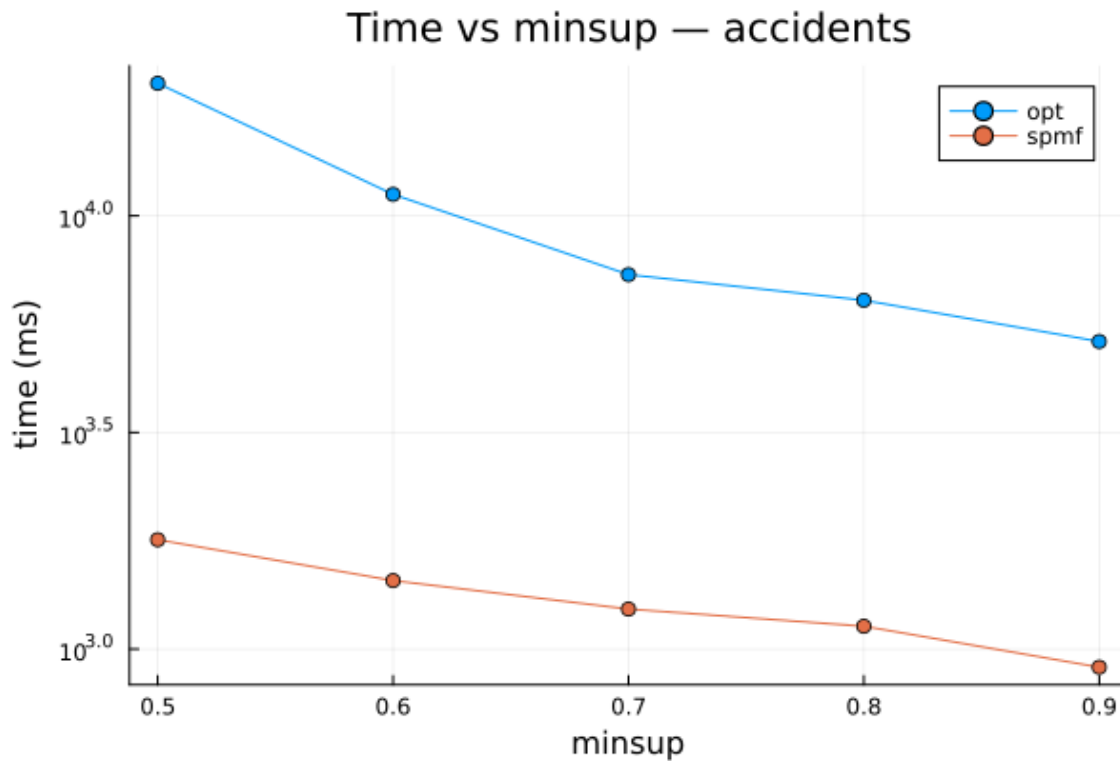
Hình 4: Thời gian chạy theo minsup trên Retail.

Hình 4 cho thấy một bức tranh ngược lại trên dữ liệu thưa. Vì Retail thưa, số tập phổ biến không lớn hơn số tối đại bao nhiêu nên phép tĩa của FP-Max ít có tác dụng, trong khi chi phí cài đặt — node dạng `String` và bước tính lại support cho từng MFI trên toàn cơ sở dữ liệu — lại lần át. Tại minsup 0.01, `opt` chậm hơn baseline (902.09 ms so với 548.92 ms). Ở minsup 0.001, `opt` mất 18959.89 ms còn SPMF chỉ 1057 ms. Đây là điểm yếu rõ nhất của cài đặt FP-Max hiện tại và là động lực cho hướng tối ưu ở cuối mục.



Hình 5: Thời gian chạy theo minsup trên T10I4D100K.

Trên T10I4D100K, vốn thừa và tổng hợp, xu hướng giống Retail: baseline nhanh hơn `opt` ở các điểm cả hai còn chạy (ví dụ minsup 0.01: 1192.31 ms so với 4347.90 ms). Lợi ích nén tiền tố và tĩa siêu tập không đủ bù chi phí cài đặt trên dữ liệu thừa.

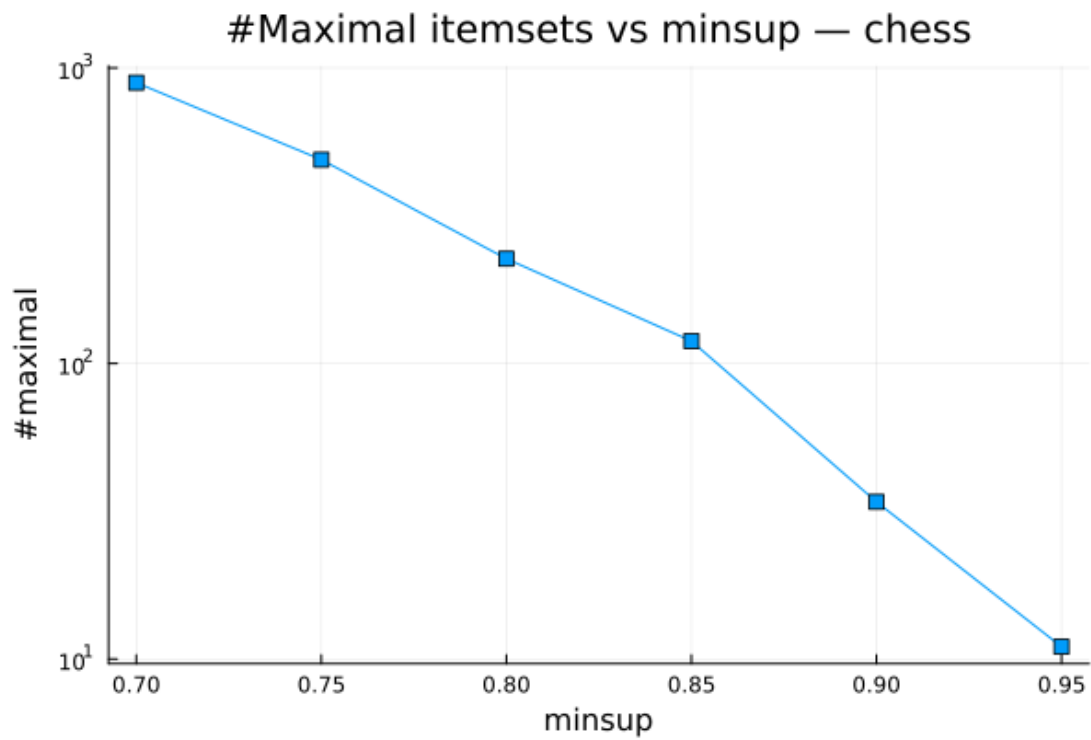


Hình 6: Thời gian chạy theo minsup trên Accidents.

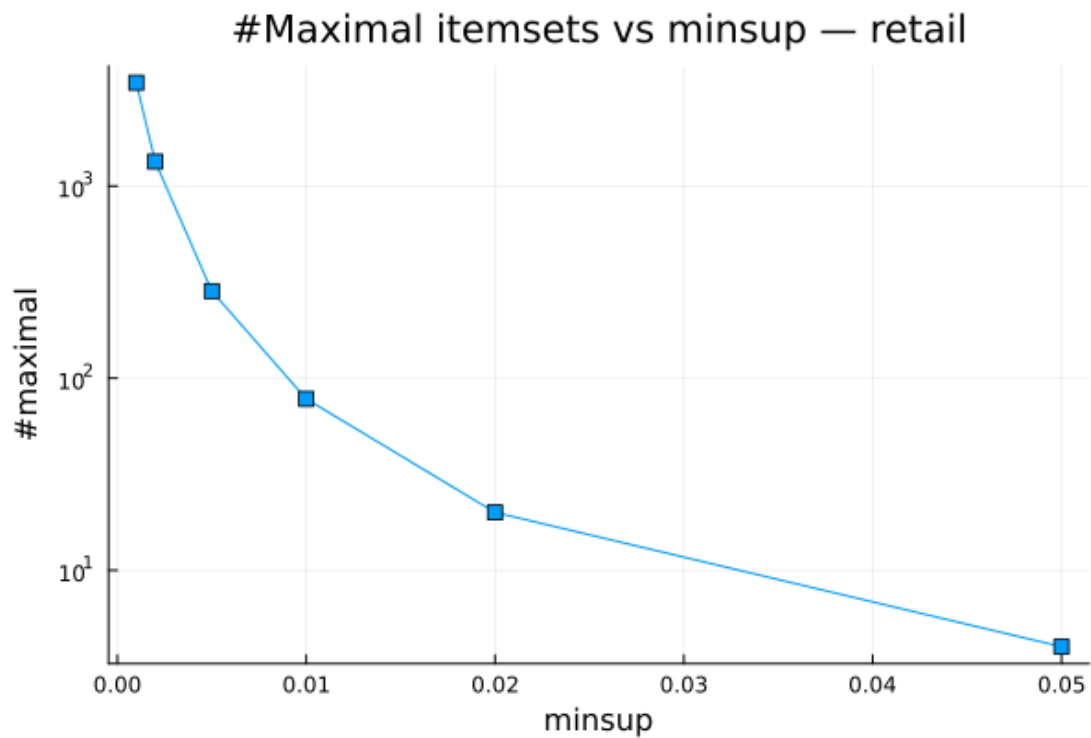
Hình 6 trình bày kết quả trên Accidents. Baseline bị bỏ qua hoàn toàn vì dữ liệu vừa lớn vừa dày khiến mine-all không khả thi — đúng vùng mà FP-Max trực tiếp phát huy giá trị. **opt** chạy được ở mọi ngưỡng từ 0.9 đến 0.5, với thời gian từ 5.13 giây (minsup 0.9, 1 tối đại) đến 20.23 giây (minsup 0.5, 216 tối đại), chậm hơn SPMF khoảng 4–11 lần. Điểm yếu chính vẫn là cài đặt cấp phát nhiều object **String** cho conditional trees và tính lại support cho từng MFI.

7.5 Số lượng tập tối đại theo minsup

Hình 7 và Hình 8 minh họa quan hệ giữa minsup và số tập tối đại đầu ra. Chess tăng từ 11 tối đại ở minsup 0.95 lên 891 tối đại ở minsup 0.70. Retail tăng từ 4 tối đại ở minsup 0.05 lên 3,452 tối đại ở minsup 0.001.



Hình 7: Số tập tối đại theo minsup trên Chess.



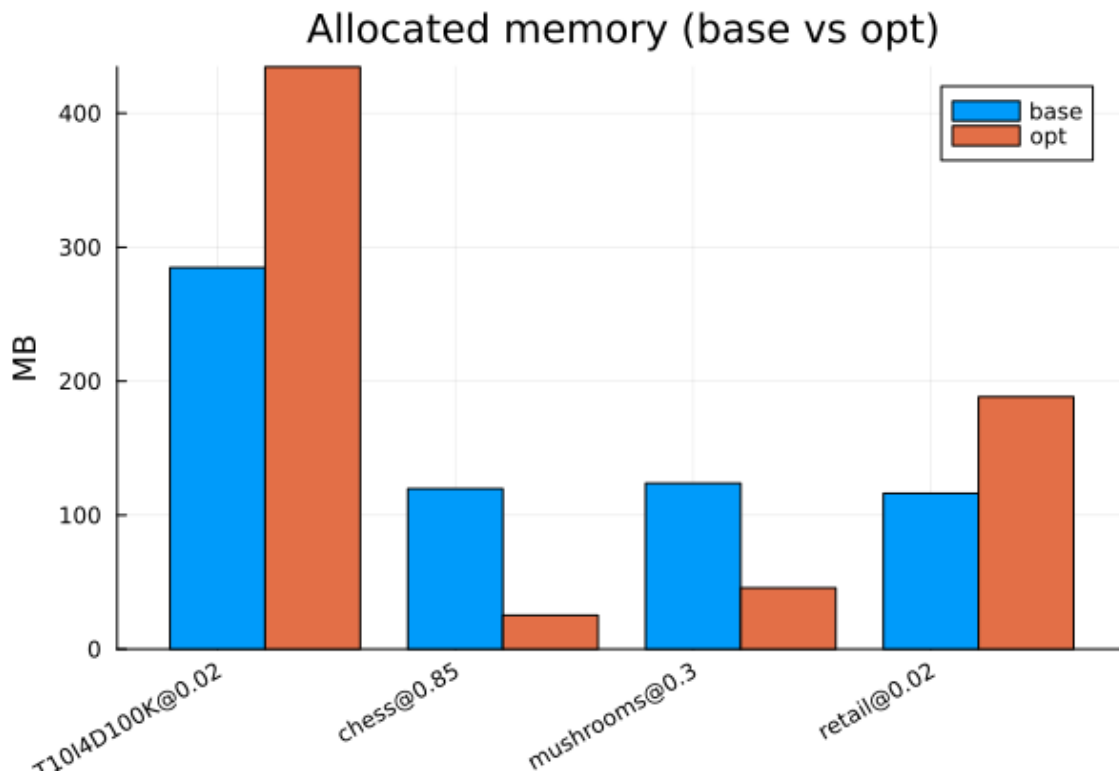
Hình 8: Số tập tối đại theo minsup trên Retail.

Sự khác biệt giữa hai hình cho thấy độ dày dữ liệu ảnh hưởng mạnh đến output size. Chess có giao

dịch dài và nhiều item đồng xuất hiện nên khi minsup giảm trong một khoảng hẹp, số tối đại tăng nhanh. Retail có nhiều item phân biệt nhưng giao dịch ngắn, vì vậy số tối đại chỉ tăng mạnh ở vùng minsup rất thấp. Đáng chú ý là số tập tối đại nhỏ hơn nhiều số tập phổ biến tương ứng: đây chính là lợi ích biểu diễn cô đọng mà FP-Max khai thác, và là lý do cài đặt trực tiếp khả thi trên Accidents trong khi baseline thì không.

7.6 Bộ nhớ

Hình 9 so sánh lượng byte cấp phát giữa baseline và FP-Max tại một minsup trung bình của từng dataset. Trên dữ liệu dày, FP-Max cấp phát ít hơn hẳn: Chess ở minsup 0.85 giảm từ 119.71 MB (base) xuống 25.23 MB (opt), khoảng 4.75 lần; Mushrooms ở minsup 0.30 giảm từ 123.79 MB xuống 45.67 MB, khoảng 2.71 lần. Ngược lại, trên dữ liệu thưa FP-Max cấp phát nhiều hơn baseline: Retail ở minsup 0.02 là 188.38 MB so với 116.27 MB, và T10I4D100K ở minsup 0.02 là 434.60 MB so với 284.74 MB. Điều này nhất quán với phân tích thời gian: phép tĩa chỉ tiết kiệm khi có nhiều tập phổ biến không tối đại để cắt.



Hình 9: Bộ nhớ cấp phát của baseline naive-maximal và FP-Max tại minsup trung bình.

Cột `alloc_bytes` phản ánh tổng lượng cấp phát trong Julia, tức áp lực lên bộ thu gom rác (GC),

chứ không phải dung lượng thường trú lớn nhất của tiến trình. Để đo trực tiếp peak memory thật, nhóm bổ sung một thực nghiệm riêng (`experiments/measure_memory.jl`): mỗi cấu hình được chạy trong một tiến trình Julia độc lập rồi đọc `Sys.maxrss()` — đỉnh resident set size (RSS) của tiến trình. Vì mỗi tiến trình mới reset RSS, giá trị `maxrss` cuối là peak của đúng lần chạy đó. Hàng *sàn* chỉ load package và dữ liệu mà không mining, cho biết phần RSS do runtime Julia chiếm. Bảng 13 tổng hợp kết quả.

Bảng 13: Peak RSS thật (`Sys.maxrss`) đo trong tiến trình riêng cho mỗi lần chạy. Cột *sàn* là cấu hình chỉ load dữ liệu, không mining.

Dataset	minsup	Sàn (MB)	base peak (MB)	opt peak (MB)
Chess	0.80	403.83	456.60	441.97
Chess	0.85	403.83	437.32	441.92
Mushrooms	0.30	420.38	449.15	448.55
Retail	0.01	463.50	500.04	534.55
T10I4D100K	0.01	461.41	791.33	842.44

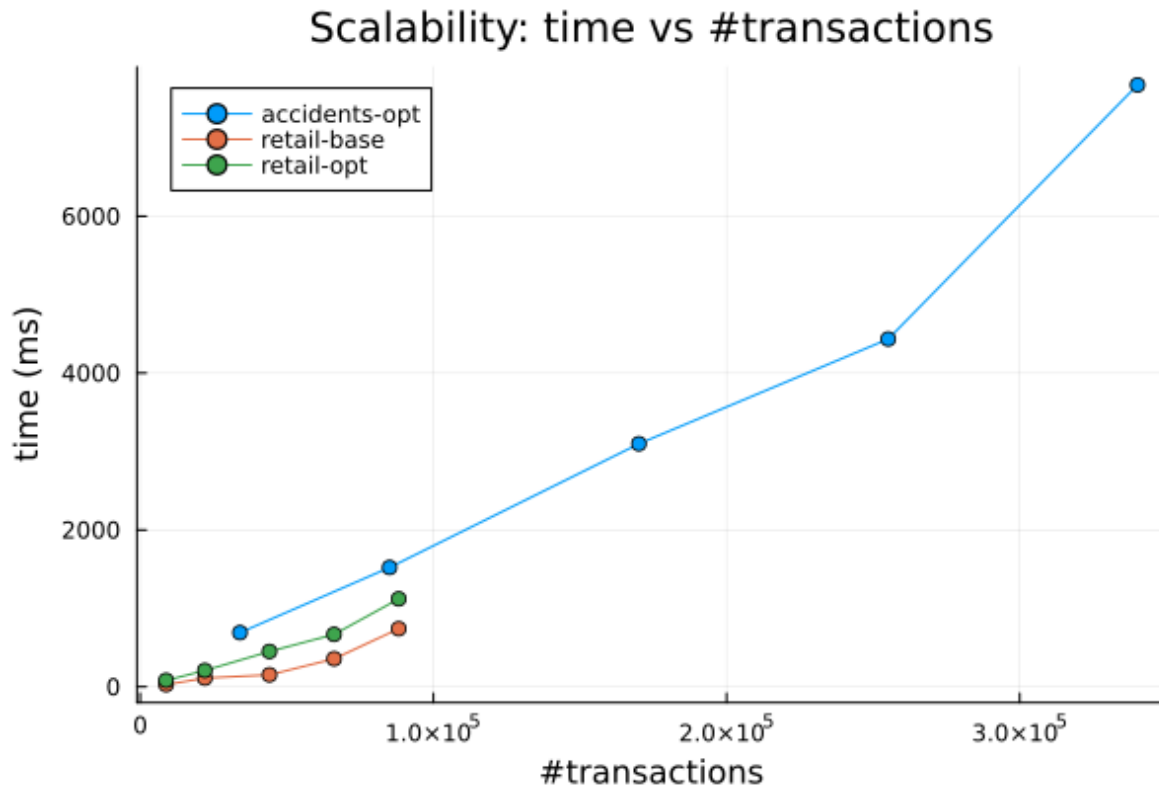
Bảng 13 cho thấy ba điểm. Thứ nhất, sàn RSS do runtime Julia, load package và dữ liệu chiếm khoảng 404–463 MB, lớn hơn nhiều phần bộ nhớ làm việc của thuật toán, nên peak RSS tuyệt đối bị chi phối bởi sàn này. Thứ hai, trên dữ liệu dày FP-Max thường trú thấp hơn hoặc xấp xỉ baseline: Chess 0.80 peak giảm từ 456.60 MB (base) xuống 441.97 MB (opt), nhất quán với xu hướng `alloc_bytes`; còn trên dữ liệu thưa (Retail, T10I4D100K) FP-Max thường trú cao hơn, đúng với việc cấp phát nhiều hơn baseline ở vùng này. Thứ ba, peak RSS thật nhỏ hơn nhiều tổng `alloc_bytes`, vì phần lớn cấp phát là tạm thời và bị GC thu hồi nên không cùng lúc thường trú. Hai chỉ số do đó bổ sung cho nhau: `alloc_bytes` đo áp lực cấp phát/GC, còn peak RSS đo dung lượng thực tế của tiến trình.

7.7 Khả năng mở rộng

Hình 10 trình bày thực nghiệm scalability, lấy các prefix 10%, 25%, 50%, 75% và 100% của Retail và Accidents. Với Accidents ở minsup 0.7, FP-Max tăng từ 690.13 ms trên 34,018 giao dịch lên 7673.25 ms trên 340,183 giao dịch; khi số giao dịch tăng 10 lần, thời gian tăng khoảng 11.1 lần — gần tuyến tính, hơi siêu tuyến do chi phí dựng và duyệt cây tăng theo độ sâu. Số tối đại dao động quanh 19–26 nên không chi phối xu hướng.

Với Retail ở minsup 0.01, FP-Max tăng từ 81.51 ms trên 8,816 giao dịch lên 1120.50 ms trên 88,162

giao dịch. Baseline naive-maximal vẫn nhanh hơn FP-Max ở mọi kích thước Retail (ví dụ 27.13 ms so với 81.51 ms ở 10%), một lần nữa khẳng định trên dữ liệu thưa chi phí cài đặt FP-Max vượt lợi ích tủa.



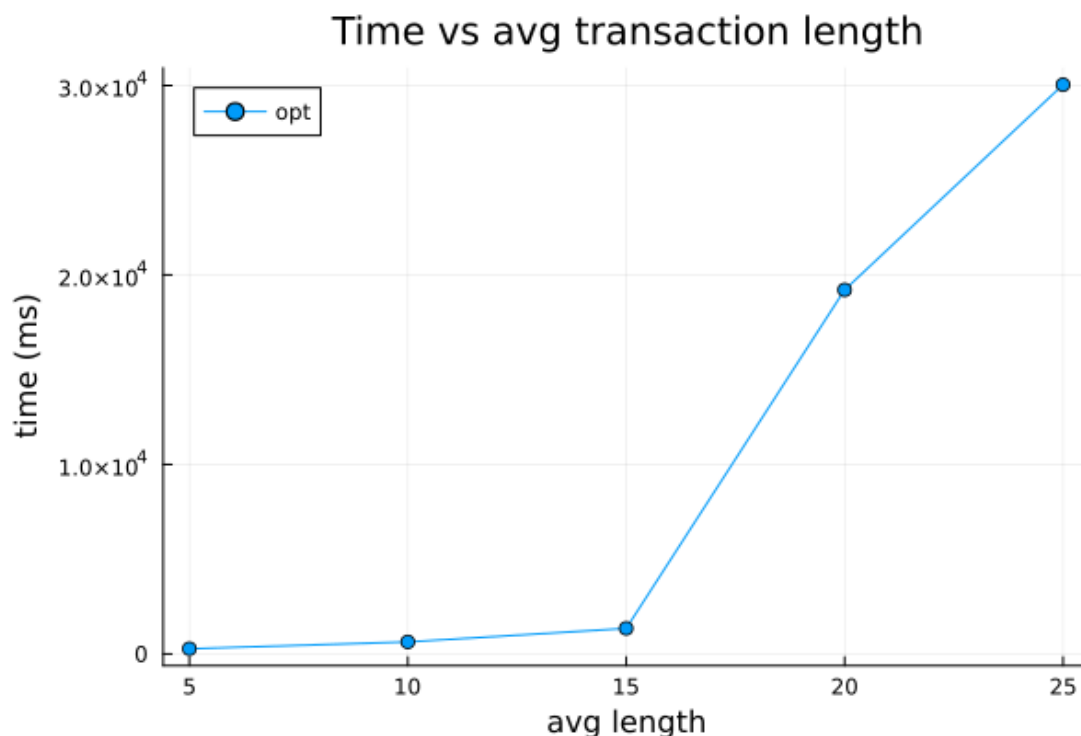
Hình 10: Khả năng mở rộng theo số lượng giao dịch.

7.8 Ảnh hưởng của độ dài giao dịch

Thí nghiệm cuối dùng dữ liệu tổng hợp 20,000 giao dịch, 100 item, minsup 0.03, và tăng độ dài giao dịch trung bình từ 5 đến 25. Bảng 14 cho thấy thời gian tăng mạnh khi độ dài trung bình đạt 20 và 25.

Bảng 14: Ảnh hưởng của độ dài giao dịch trung bình lên thời gian chạy FP-Max.

AvgLen	Time ms	#Tối đại
5	276.31	100
10	636.31	100
15	1349.64	100
20	19213.50	4,950
25	30039.70	4,950



Hình 11: Thời gian chạy FP-Max khi tăng độ dài giao dịch trung bình.

Kết quả này phù hợp với lý thuyết: giao dịch dài làm tăng xác suất các item cùng xuất hiện, làm FP-tree dày hơn và số tập tối đại tăng. Khi output size nhảy từ 100 lên 4,950 tối đại (giữa độ dài 15 và 20), thời gian cũng nhảy từ 1349.64 ms lên 19213.50 ms, tức tăng theo bậc lớn hơn nhiều so với mức tăng tuyến tính của độ dài giao dịch; Hình 11 minh họa rõ bước nhảy này, đúng với cận mũ ở trường hợp xấu khi số tối đại M bùng nổ.

7.9 Nhận xét và hướng tối ưu tiếp theo

Thực nghiệm cho thấy cài đặt FP-Max của nhóm đúng so với SPMF FPMax trên mọi cấu hình kiểm tra. Điểm mạnh quyết định là FP-Max trực tiếp hoàn tất trong những vùng mà baseline naive-maximal không khả thi — toàn bộ Accidents và các ngưỡng thấp trên dữ liệu dày — nhờ chỉ giữ biểu diễn tối đại và tĩa theo siêu tập. Trên dữ liệu dày như Chess và Mushrooms ở minsup thấp, FP-Max nhanh hơn baseline nhiều lần và cấp phát ít hơn rõ rệt.

Điểm yếu có hai mức. So với baseline, FP-Max chậm hơn trên dữ liệu thưa (Retail, T10I4D100K) vì phép tĩa ít tác dụng còn chi phí cài đặt cao. So với SPMF, FP-Max chậm hơn trên hầu hết cấu hình lớn. Nguyên nhân chính nằm ở cài đặt: FP-Max hiện dùng FP-tree với node `String` và tính

lại support cho từng MFI bằng một lần quét toàn cơ sở dữ liệu.

Từ đó, hai hướng tối ưu cụ thể có thể làm tiếp. Thứ nhất, chuyển FP-Max sang dùng cấu trúc cây mã hoá số nguyên như `FPGrowthOpt` (header và children typed, @inbounds) để loại chi phí `String` trong hot loop. Thứ hai, mang support theo trong quá trình khai thác để bỏ hẳn bước tính lại support cho từng MFI ở cuối, vốn tốn một lần quét dữ liệu mỗi tối đại. Ngoài ra, có thể thay danh sách MFI tuyến tính bằng một MFI-tree để phép kiểm tra siêu tập có chi phí dưới tuyến tính, và song song hoá các nhánh conditional tree độc lập.

8 Ứng dụng

8.1 Bài toán phân tích giỏ hàng

Ứng dụng được chọn là phân tích giỏ hàng (Market Basket Analysis). Với mỗi giao dịch là một hoá đơn mua hàng, frequent itemsets cho biết các nhóm sản phẩm thường xuất hiện cùng nhau. Từ các itemset phổ biến, ta sinh luật kết hợp dạng $X \Rightarrow Y$, trong đó X là vế trái, Y là vế phải và $X \cap Y = \emptyset$.

Ba chỉ số được dùng để đánh giá luật là support, confidence và lift:

$$\text{conf}(X \Rightarrow Y) = \frac{\text{sup}(X \cup Y)}{\text{sup}(X)}$$

và

$$\text{lift}(X \Rightarrow Y) = \frac{\text{conf}(X \Rightarrow Y)}{\text{sup}_{rel}(Y)}.$$

Confidence đo xác suất mua Y khi đã mua X . Lift đo mức tăng so với xác suất mua Y độc lập [Brin et al. \(1997\)](#). Nếu lift lớn hơn 1, X và Y có xu hướng xuất hiện cùng nhau nhiều hơn kỳ vọng ngẫu nhiên [Agrawal et al. \(1993\)](#).

8.2 Dữ liệu và thiết lập

Nhóm sử dụng tập `data/groceries.txt`, gồm 9,835 giao dịch và 169 mặt hàng. Độ dài giao dịch trung bình là 4.41 item, giao dịch dài nhất có 32 item. Vì tên mặt hàng trong Groceries có thể chứa khoảng trắng, dữ liệu được tách theo dấu phẩy trước khi đưa vào bộ máy khai thác của nhóm. Cần lưu ý rằng sinh luật kết hợp đòi hỏi support của mọi itemset liên quan (vế trái X , vế phải Y , và

hợp $X \cup Y$) để tính confidence và lift, trong khi tập tối đại do FP-Max trả về không lưu lại support của các tập con. Do đó phần ứng dụng này dùng bộ máy FP-Growth khai thác đầy đủ tập phổ biến (cùng cài đặt mà baseline maximal của nhóm tái sử dụng), còn FP-Max vẫn là thuật toán được chọn của đồ án cho biểu diễn mẫu tối đại cô đọng. Thiết lập khai thác là:

- $\text{minsup} = 0.01$, tương đương ít nhất $\lceil 0.01 \times 9835 \rceil = 99$ giao dịch.
- $\text{minconf} = 0.2$.
- Luật được lọc theo $\text{lift} > 1$ rồi sắp theo lift giảm dần; nếu lift bằng nhau thì sắp theo thứ tự bảng chữ cái của vế trái, sau đó của vế phải.

Với thiết lập này, thuật toán tìm được 333 frequent itemsets và sinh 231 luật thoả minconf và có lift lớn hơn 1.

8.3 Top-10 luật theo lift

Bảng 15 trình bày 10 luật có lift cao nhất. Support trong bảng là support tương đối.

Bảng 15: Top-10 luật kết hợp trên Groceries theo lift, với $\text{minsup} = 0.01$ và $\text{minconf} = 0.2$.

Vế trái X	Vế phải Y	Support	Confidence	Lift
citrus fruit + other vegetables	root vegetables	0.0104	0.3592	3.2950
other vegetables + yogurt	whipped/sour cream	0.0102	0.2342	3.2671
other vegetables + tropical fruit	root vegetables	0.0123	0.3428	3.1448
beef	root vegetables	0.0174	0.3314	3.0404
citrus fruit + root vegetables	other vegetables	0.0104	0.5862	3.0296
root vegetables + tropical fruit	other vegetables	0.0123	0.5845	3.0210
other vegetables + whole milk	root vegetables	0.0232	0.3098	2.8421
root vegetables	other vegetables + whole milk	0.0232	0.2127	2.8421
butter	other vegetables + whole milk	0.0115	0.2073	2.7706
curd + whole milk	yogurt	0.0101	0.3852	2.7614

8.4 Diễn giải kinh doanh

Các luật mạnh nhất tập trung quanh nhóm rau củ, sữa và sữa chua. Luật “citrus fruit + other vegetables $\Rightarrow$ root vegetables” có lift 3.2950, nghĩa là trong các giỏ hàng đã có trái cây họ cam

chanh và rau khác, khả năng xuất hiện root vegetables cao hơn khoảng 3.3 lần so với tần suất nền của root vegetables. Đây là tín hiệu phù hợp cho gợi ý mua kèm hoặc bố trí kệ gần nhau giữa nhóm rau củ và trái cây tươi.

Luật “beef $\Rightarrow$ root vegetables” có confidence 0.3314 và lift 3.0404. Về mặt kinh doanh, người mua thịt bò có xu hướng mua thêm rau củ dùng cho món hầm, súp hoặc bữa ăn chính. Siêu thị có thể dùng luật này để thiết kế combo nguyên liệu hoặc coupon giảm giá chéo.

Nhóm luật liên quan tới *whole milk*, *curd* và *yogurt* cho thấy các sản phẩm sữa có quan hệ đồng mua rõ. Ví dụ “curd + whole milk $\Rightarrow$ yogurt” có confidence 0.3852 và lift 2.7614. Luật này phù hợp cho gợi ý trong giỏ hàng trực tuyến: khi khách đã chọn curd và whole milk, hệ thống có thể đề xuất yogurt như một sản phẩm bổ sung.

Vì ngưỡng minconf đặt thấp (0.2), top theo lift còn chứa một số luật có confidence khiêm tốn, ví dụ “other vegetables + yogurt $\Rightarrow$ whipped/sour cream” chỉ có confidence 0.2342 dù lift đạt 3.2671. Lift cao trong trường hợp này phản ánh whipped/sour cream là mặt hàng hiếm, nên một mức tăng nhỏ về xác suất đã đẩy lift lên. Ngoài ra một vài luật có vẻ phải gồm hai item, như “root vegetables $\Rightarrow$ other vegetables + whole milk”. Khi đọc các luật này cần xét confidence cùng lift, không chỉ riêng lift.

Một điểm cần lưu ý là support của các luật top lift chỉ quanh 1–2.3%. Các luật này có ý nghĩa định hướng gợi ý theo ngữ cảnh, nhưng không nên xem là luật bao phủ phần lớn khách hàng. Nếu mục tiêu là khuyến mãi diện rộng, cần cân bằng lift với support tuyệt đối và lợi nhuận của từng sản phẩm.

9 Kết luận

Đồ án đã hoàn thành việc nghiên cứu, cài đặt và đánh giá thuật toán FP-Max cho bài toán khai thác tập phổ biến tối đại. Về lý thuyết, báo cáo trình bày các định nghĩa cốt lõi của FIM, tính chất Apriori, cấu trúc FP-tree làm nền, và phần trọng tâm về FP-Max: danh sách MFI, phép tĩa theo siêu tập, shortcut single-path và phân tích độ phức tạp. Hai ví dụ tay minh hoạ FP-Max trên cả trường hợp cơ sở (cập nhật danh sách MFI) lẫn tình huống đặc biệt (phép tĩa bỏ qua nguyên một nhánh).

Về cài đặt, nhóm xây dựng package Julia `FrequentItemsetMining` gồm bộ máy nền FP-Growth (bản cơ bản và bản tối ưu mã hoá số nguyên), một baseline naive-maximal khai thác toàn bộ tập

phổ biến rồi lọc tối đại, và FP-Max khai thác trực tiếp có danh sách MFI và tĩa theo siêu tập. Output được giữ tương thích với định dạng SPMF để thuận tiện kiểm thử và đối chiếu.

Về thực nghiệm, kết quả correctness đạt `match_ratio = 1.0` và `support_mismatch = 0` so với SPMF FPMax trên mọi cấu hình benchmark được kiểm tra. Điểm mạnh quyết định của FP-Max trực tiếp là chạy được trong những vùng mà baseline naive-maximal không khả thi — toàn bộ Accidents và các ngưỡng thấp trên dữ liệu dày — nhờ chỉ giữ biểu diễn tối đại và tĩa không gian tìm kiếm. Trên dữ liệu dày, FP-Max còn nhanh hơn baseline nhiều lần, ví dụ khoảng 8.68 lần trên Chess tại minsup 0.8 và 4.84 lần trên Mushrooms tại minsup 0.25. Tuy vậy, trên dữ liệu thưa (Retail, T10I4D100K) FP-Max chậm hơn baseline vì phép tĩa ít tác dụng còn chi phí cài đặt cao, và nhìn chung vẫn chậm hơn SPMF.

Phần ứng dụng trên Groceries dùng bộ máy khai thác đầy đủ tập phổ biến (vì sinh luật cần support của mọi itemset) để tạo các luật kết hợp có ý nghĩa thực tế, chẳng hạn các luật liên quan tới *root vegetables*, *other vegetables*, *whole milk* và *yogurt*. Các luật này có thể hỗ trợ gợi ý sản phẩm, bố trí kệ hàng và thiết kế combo.

Các hạn chế còn lại của đồ án gồm: cài đặt FP-Max hiện dùng FP-tree với node **String** và tính lại support cho từng MFI bằng một lần quét toàn cơ sở dữ liệu, khiến nó chậm trên dữ liệu thưa; baseline naive-maximal phải skip nhiều cấu hình do nguy cơ bùng nổ. Trong tương lai, nhóm có thể chuyển FP-Max sang cấu trúc cây mã hoá số nguyên như bộ máy tối ưu, mang support theo trong quá trình khai thác để bỏ bước tính lại, thay danh sách MFI tuyến tính bằng một MFI-tree cho phép kiểm tra siêu tập dưới tuyến tính, và song song hoá các nhánh conditional tree độc lập.

Tài liệu

- Agrawal, R., Imielinski, T., and Swami, A. (1993). Mining association rules between sets of items in large databases. In *Proceedings of the 1993 ACM SIGMOD International Conference on Management of Data*, pages 207–216.
- Agrawal, R. and Srikant, R. (1994). Fast algorithms for mining association rules in large databases. In *Proceedings of the 20th International Conference on Very Large Data Bases (VLDB)*, pages 487–499.
- Brin, S., Motwani, R., Ullman, J. D., and Tsur, S. (1997). Dynamic itemset counting and implication rules for market basket data. In *Proceedings of the 1997 ACM SIGMOD International Conference on Management of Data*, pages 255–264.
- Fournier-Viger, P., Gomariz, A., Gueniche, T., Soltani, A., Wu, C.-W., and Tseng, V. S. (2014). SPMF: A java open-source pattern mining library. *Journal of Machine Learning Research*, 15:3389–3393.
- Goethals, B. and Zaki, M. J. (2003). Frequent itemset mining dataset repository. FIMI Workshop, <http://fimi.uantwerpen.be/data/>.
- Grahne, G. and Zhu, J. (2003). High performance mining of maximal frequent itemsets. In *Proceedings of the Workshop on Frequent Itemset Mining Implementations*.
- Han, J., Pei, J., and Yin, Y. (2000). Mining frequent patterns without candidate generation. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, pages 1–12.
- Zaki, M. J. (2000). Scalable algorithms for association mining. *IEEE Transactions on Knowledge and Data Engineering*, 12(3):372–390.

A Phụ lục: Hướng dẫn tái lập

Phụ lục này tóm tắt các bước tái lập toàn bộ mã nguồn, kiểm thử và thực nghiệm của đồ án. Nội dung tương ứng với tệp `README.md` trong kho mã nguồn; mọi lệnh đều chạy từ thư mục gốc của repo. Mã nguồn đầy đủ tại github.com/pineapplechoco/dma-lab2.

A.1 Cấu trúc thư mục

- `src/`: package Julia `FrequentItemsetMining` (FP-Max và baseline naive-maximal, bộ máy nền FP-Growth cơ bản và tối ưu, sinh luật kết hợp, I/O).
- `main.jl`: giao diện dòng lệnh.
- `test/`: bộ kiểm thử correctness và benchmark.
- `experiments/`: harness thực nghiệm Chương 7 và ứng dụng Chương 8.
- `data/`: CSDL toy, benchmark và Groceries.
- `docs/Report.pdf`: báo cáo.
- `Project.toml`, `Manifest.toml`: khai báo và khoá phiên bản phụ thuộc.

Đồ án không sử dụng bất kỳ thư viện FIM có sẵn nào; SPMF chỉ được gọi như một công cụ hộp đen để đối chiếu kết quả ở Chương 7.

A.2 Môi trường và cài đặt

Yêu cầu Julia ≥ 1.9 (đã kiểm thử trên Julia 1.12). Cách khuyến nghị là dùng `juliaup` — trình quản lý phiên bản chính thức của Julia:

```
1 curl -fsSL https://install.julialang.org | sh
2 juliaup add 1.12          # optional: pin the tested version
3 julia --version           # expected: julia version 1.12.x (or >= 1.9)
```

Listing 6: Cài Julia bằng `juliaup` (Linux/macOS) và ghim phiên bản.

Trên Windows có thể cài từ Microsoft Store (tìm “Julia”) hoặc chạy `winget install julia -s msstore`. Sau khi có Julia, cài phụ thuộc theo `Manifest.toml` đã khoá để đảm bảo tái lập:

```
1 julia --project=. -e 'using Pkg; Pkg.instantiate()'
```

Listing 7: Cài đặt phụ thuộc.

A.3 Chạy chương trình

Giao diện dòng lệnh nhận đường dẫn dữ liệu, ngưỡng minsup tương đối, thuật toán và đường dẫn output (tùy chọn):

```
1 julia --project=. main.jl <input_path> <minsup> [fp-growth|fp-growth-opt|fp-max|  
rules] [output_path]
```

Listing 8: Cú pháp dòng lệnh.

Ví dụ khai thác tập phổ biến tối đại bằng FP-Max trên CSDL toy với minsup 0.6 (kết quả $\{1, 3\} : 3$, $\{2, 3, 5\} : 3$):

```
1 julia --project=. main.jl data/toy/test_1.txt 0.6 fp-max output.txt
```

Listing 9: Khai thác tập tối đại bằng FP-Max.

Đầu vào theo định dạng SPMF (mỗi giao dịch một dòng, item cách nhau bằng khoảng trắng; parser cũng chấp nhận dòng có dấu phẩy). Đầu ra theo dạng `item1 item2 ... #SUP: support_count`. Chế độ `rules` sinh trực tiếp luật kết hợp (lift > 1, sắp theo lift giảm dần) từ tập phổ biến. Ngưỡng confidence đặt qua biến môi trường MINCONF (mặc định 0.2), số luật in ra qua TOPK (mặc định 10). Với dữ liệu mà tên item chứa khoảng trắng (ví dụ Groceries), đặt `SEP=comma` để parser chỉ tách theo dấu phẩy:

```
1 SEP=comma julia --project=. main.jl data/groceries.txt 0.01 rules rules.csv
```

Listing 10: Sinh luật kết hợp trên Groceries.

Lệnh trên tái tạo đúng kết quả phần ứng dụng: 333 frequent itemsets và 231 luật có lift lớn hơn 1.

A.4 Kiểm thử

```
1 julia --project=. test/runtests.jl
```

Listing 11: Chạy bộ kiểm thử tự động.

Bộ kiểm thử gồm: kiểm tra parser ba định dạng; FP-Growth và FP-Max trên CSDL toy ở Chương 5; đối chiếu bản tối ưu với bản cơ bản trên năm CSDL (hai toy và ba benchmark: chess, mushrooms, retail); kiểm tra sinh luật kết hợp; và benchmark đối chiếu base/opt trên chess. Các tỉ lệ tốc độ/bộ nhớ in trong dòng [bench] là số đo on-the-fly, phụ thuộc máy và lần chạy (JIT warm-up, GC) nên có thể dao động, không nhất thiết trùng với số liệu cố định trong báo cáo.

A.5 Tái lập thực nghiệm Chương 4

Thực nghiệm cần SPMF (làm chuẩn đối chiếu, yêu cầu Java ≥ 8) và đầy đủ năm tập benchmark.

1. Tải SPMF jar (bị .gitignore vì nặng):

```
1 bash experiments/download_spmf.sh
```

2. Tải dataset. Bốn tập chess, mushrooms, retail, T10I4D100K đã có sẵn trong data/benchmark/; bổ sung accidents.txt (> 25 MB, không đưa vào repo) từ Google Drive vào data/benchmark/:

[Thư mục dữ liệu trên Google Drive.](#)

3. Chạy thực nghiệm (sinh các file CSV trong experiments/results/):

```
1 julia --project=. experiments/run_experiments.jl
```

Có thể giới hạn tập dữ liệu qua biến môi trường, ví dụ bỏ qua accidents:

```
1 EXP_DATASETS=chess,mushrooms,retail,T10I4D100K julia --project=. experiments/
  run_experiments.jl
```

4. Đo peak RSS thật (sinh experiments/results/memory.csv):

```
1 julia --project=. experiments/measure_memory.jl
```

5. Sinh biểu đồ (vào experiments/figures/):

```
1 julia --project=. experiments/make_figures.jl
```

A.6 Tái lập ứng dụng Chương 5

Sinh luật kết hợp trên tập Groceries (minsup 0.01, minconf 0.2), in top-10 luật theo lift và ghi experiments/results/rules_groceries.csv:

```
1 julia --project=. experiments/application.jl
```

Listing 12: Phân tích giỏ hàng.

Kết quả mong đợi: 333 frequent itemsets và 231 luật có lift lớn hơn 1. Kết quả này cũng đạt được trực tiếp qua chế độ `rules` của `main.jl` như ở mục Chạy chương trình.

A.7 Tính tái lập

Mọi dữ liệu tổng hợp dùng seed cố định (`experiments/datagen.jl`, seed 42); `Manifest.toml` khoá toàn bộ phiên bản phụ thuộc. Số đếm (`#frequent itemsets`, `#luật`) và kết quả `correctness` là tất định nên chạy lại cho ra cùng giá trị; riêng các số đo thời gian và peak RSS có thể dao động theo máy và lần chạy do JIT, GC và tải hệ thống.